

Génération de code temps-réel

T. Carle
2022-2027



Introduction

- Systèmes temps-réel critiques
 - Échéances à respecter absolument
 - Capacité à garantir le respect des échéances à l'avance
 - Code simple et rigide sur les dates => + simple à certifier
- Systèmes cycliques
 - Systèmes réactifs
 - Systèmes multi-périodiques
- Utilisation de tables de réservation statiques
 - IMA (ARINC 653), AUTOSAR

Introduction

- Adéquation Algorithme Architecture

Fonctions
Tâches
Runnables
Noeuds scade
Schéma de
comm.

Spécification
fonctionnelle

Spécification
non-fonctionnelle

HW
WCETs
Contraintes TR
(périodes,
échéances)

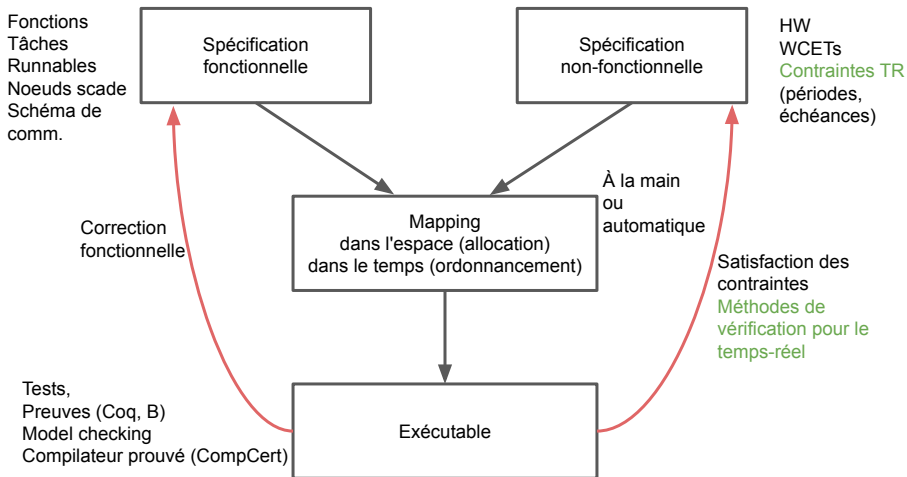
Mapping
dans l'espace (allocation)
dans le temps (ordonnancement)

À la main
ou
automatique

Exécutable

Introduction

- Adéquation Algorithme Architecture



Introduction

- Ingénierie d'un système temps-réel
 - **Spécification** : exigences fonctionnelles et non fonctionnelles
 - Fonctionnel : Qu'est-ce que le système calcule? **Définition des tâches/noeuds**
 - Non-fonctionnel : Sur quel matériel? Avec quelles **contraintes temporelles**?
 - **Implémentation** : production du système sous contraintes
 - Choix du **langage** : asynchrone (ADA, C) vs synchrone (Lustre, SCADE)
 - **Exécutif cyclique** ou implem **concurrente**. Lier les **tâches** de spécification aux **threads** d'exécution
 - Cible code **statique** ou configuration d'**ordonnanceur/dispatcheur**
 - **Vérification** : vérifier les contraintes sur le système final
 - Fonctionnel : pas dans ce cours
 - Non-fonctionnel : **analyse d'ordonnançabilité, tables de réservations**

Introduction

- Partie 1 : spécification et modèles classiques
- Partie 2 : génération d'exécutifs cycliques
- Partie 3 : tables d'ordonnancement. List scheduling. Rate monotonic statique, analyse WCRT.
- Partie 4 : génération de code synchronisé. Implémentation des préemptions.
- Partie 5 : problèmes récents et/ou ouverts. CRPD, multi-coeur.

Spécification

- Sources des contraintes temps-réel
 - **Lois Physiques**
 - Objets en mouvement, inertie
 - => **Contraintes de bout en bout, échéances**
 - **Automatique** (théorie du contrôle)
 - => **Fréquences d'exécution/périodes**
 - **Limitations matérielles**
 - Fréquence d'échantillonnage des capteurs
 - Fréquence des actionneurs
 - => **Périodes**

Spécification

Exemple : un système de contrôle automobile simplifié

- Exigences fonctionnelles :
 - Unité de contrôle moteur (ECU)
 - Echantillonnage d'entrées depuis un capteur d'angle de pédale, un capteur de vitesse et un capteur de rotation moteur
 - Ecriture dans un contrôleur dans le moteur pour ajuster la vitesse de rotation
 - Unité de contrôle d'airbag
 - Echantillonnage d'entrées depuis un détecteur de collision
 - Déclenchement de l'airbag si nécessaire
- Exigences temps-réel
 - Période d'échantillonnage des capteurs
 - Angle de pédale, vitesse : 10ms
 - Rotation du moteur : 20ms
 - Détection de collision : 60ms
 - Périodes des actionneurs
 - Fonction ECU (automatique) : 40ms
 - Fonction airbag (corps en mouvement) : 60ms

Spécification

Abstraction la plus courante : **Tâches temps-réel**

- Les **tâches** représentent les unités fonctionnelles du système (e.g. une fonction ou un groupe de fonctions)
- Tâches **périodiques** : très utilisées en temps-réel
 - **Période** (T_i) : durée entre deux **activations** (releases) successives de la tâche τ_i
 - **Echéance relative** (D_i) : durée entre une activation d'une tâche et la date à laquelle son exécution doit être terminée
 - Lorsque $D_i = T_i$ on omet simplement D_i (**deadline implicite**)
 - **Budget** de temps d'exécution (C_i) : temps alloué à la tâche pour s'exécuter (en général WCET)
- La périodicité induit de multiples **instances** des tâches (**jobs**)
 - Date d'**arrivée** ($A_{i,j}$) : la date d'activation du j^{eme} job de τ_i i.e. la date à partir de laquelle le job peut être ordonnancé (release date)

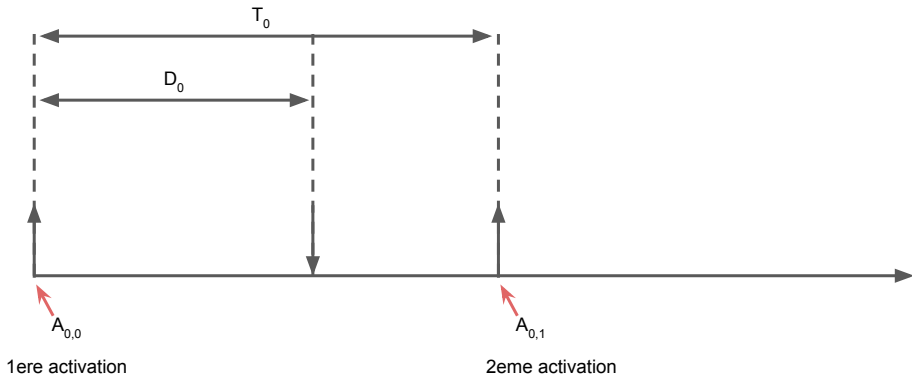
Spécification

Abstraction la plus courante : **Tâches temps-réel**

- Tâches **sporadiques**
 - Pas périodiques mais on connaît *a priori* une durée minimale entre l'arrivée de deux jobs successifs
 - e.g. Routines de tolérance aux fautes, certaines tâches d'échantillonnage de capteurs
- Tâches **apériodiques**
 - Peuvent s'activer n'importe quand => rend la vérification/analyse d'ordonnançabilité difficile
 - Peuvent être traitées par des **tâches "serveurs"** périodiques

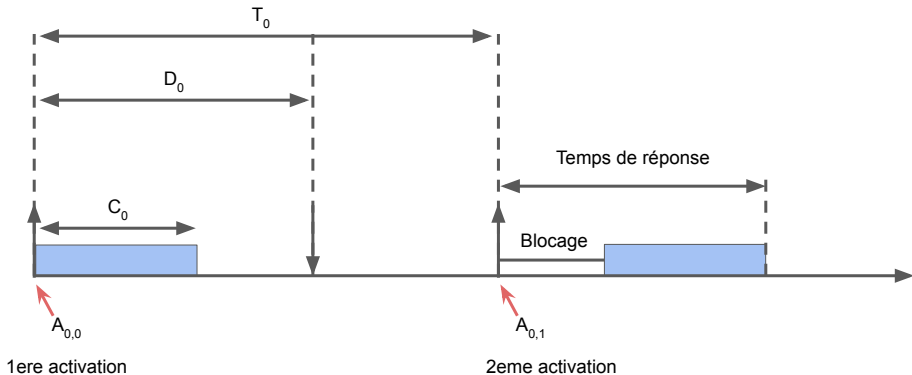
Spécification

Exemple : $\tau_0 : T_0 = 10, D_0 = 6, C_0 = 3$



Spécification

Exemple : $\tau_0 : T_0 = 10, D_0 = 6, C_0 = 3$



Spécification

- Deux utilisations classiques :
 - **Système multipériodique**
 - Les tâches peuvent avoir des **périodes différentes**
 - En général, les communications restent **implicites**
 - Très forte affinité avec les solutions basées sur les RTOS (ordonnancement dynamique)
 - **Système monopériodique communicant**
 - **Une seule période dans le système**
 - **Communications explicites**
 - Pas de notion de rythmes/périodes différents entre les tâches
 - Très forte affinité avec les solutions basées sur la compilation (ordonnancement statique)
- Des mélanges possibles :
 - Multipériode + communications explicites : difficile de spécifier proprement les communications à la granularité des jobs
 - **Multipériode étendu sur hyperpériode** : instancier les jobs pour se ramener en mono-période et expliciter les communications
 - Compromis intéressant

Spécification

Exemple : un système de contrôle automobile simplifié

- Exigences fonctionnelles :
 - Unité de contrôle moteur (ECU)
 - Echantillonnage d'entrées depuis un capteur d'angle de pédale, un capteur de vitesse et un capteur de rotation moteur
 - Ecriture dans un contrôleur dans le moteur pour ajuste la vitesse de rotation
 - Unité de contrôle d'airbag
 - Echantillonnage d'entrées depuis un détecteur de collision
 - Déclenchement de l'airbag si nécessaire
- Exigences temps-réel
 - Période d'échantillonnage des capteurs
 - Angle de pédale, vitesse : 10ms
 - Rotation du moteur : 20ms
 - Détection de collision : 60ms
 - Périodes des actionneurs
 - Fonction ECU (automatique) : 40ms
 - Fonction airbag (corps en mouvement) : 60ms

Comment capturer les exigences dans le modèle de tâches périodiques ?

Spécification

- Modèle 1 : **1 tâche par fonctionnalité** (tâches gros-grain)
 - Chaque tâche lit ses entrées, calcule les corrections et écrit vers les actionneurs
 - Conception niveau système très simple
 - Peu de tâches
 - Pas de communication (même implicite) entre tâches
 - Complexité cachée dans les tâches
 - **Périodes d'échantillonnages/actionneurs différentes**
 - Les tâches peuvent être plus complexes (voire impossibles) à implémenter
 - Les WCETs sont plus difficiles à obtenir
 - **Abstraction forte**, qui mène souvent à du pessimisme lors de la phase de validation
 - **Peu de place pour des optimisations au niveau système**

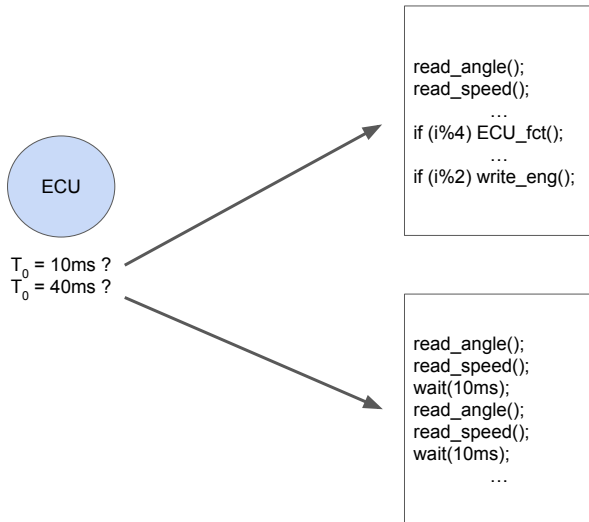


$T_0 = 10\text{ms} ?$
 $T_0 = 40\text{ms} ?$



$T_1 = 60\text{ms}$

Spécification

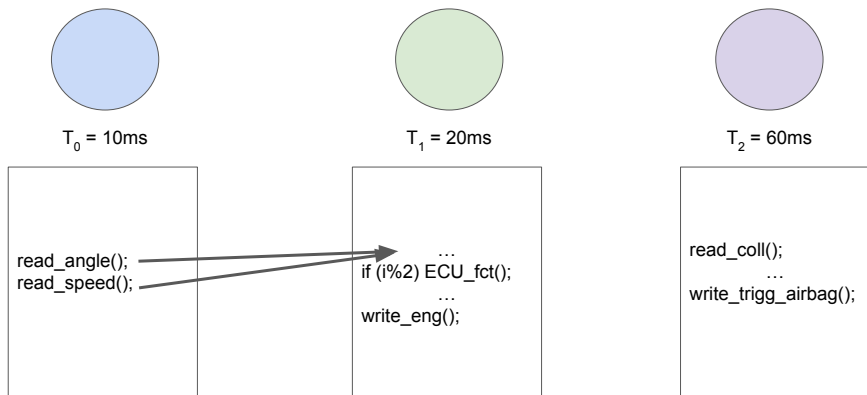


Spécification

Modèle 2 : **1 tâche par vitesse souhaitée** (tâches "porteuses")

- On choisit un certain nombre de **périodes**, et on crée une tâche par période
- On remplit les tâches avec des appels aux fonctions s'activant sur ces périodes (ou multiples de périodes)
- Conception niveau système modérément simple
 - Peu de tâches
 - Il faut sélectionner les "bonnes" périodes en fonction du logiciel et du matériel
 - Communications implicites ou explicites entre tâches, qui peuvent poser problème
- Complexité dans les tâches
 - Communications intra-tâches => respecter les **dépendances**
 - Communications inter-tâches => **synchronisations**
 - WCET potentiellement difficiles à obtenir
- Aucune place pour de l'optimisation niveau système (RTOS + ordonnancement dynamique)

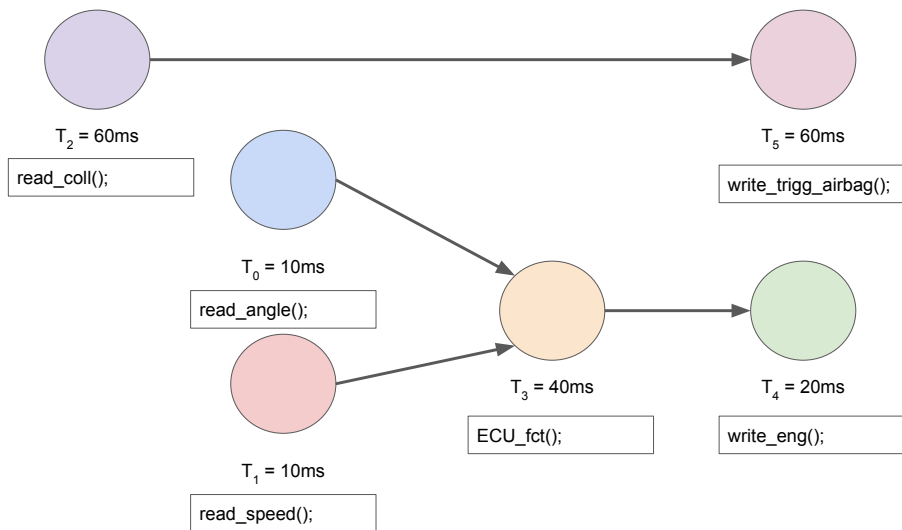
Spécification



Modèle 3 : **1 tâche par appel de fonction** (tâches fin-grain)

- On crée une tâche par fonction "basique"
- Conception niveau **système plus complexe** mais **tâches plus simples**
 - **Pas de communications/synchronisations internes** aux tâches
 - Communications inter-tâches explicites ou implicites
- Abstraction moins forte
 - Analyse système plus dure, mais ouvre la voie à des **optimisations**
 - Surestimation WCET plus faible quand on analyse les tâches

Spécification



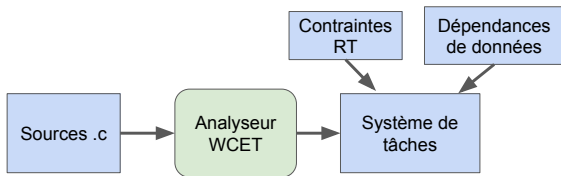
Spécification

- Dans la pratique, mélange des différents modèles selon :
 - Le système ciblé
 - Les pratiques/standards du domaine
 - Les pratiques de l'entreprise, le code hérité
- Le choix du modèle :
 - Transforme les exigences en contraintes
 - Les contraintes peuvent être plus fortes que les exigences (**coût de l'abstraction**) : e.g. faire tourner la tâche ECU du modèle 1 à 10ms
 - Peut être vu comme le premier pas dans l'implémentation (la limite entre spécification et implémentation est floue)
 - e.g. le choix entre les 3 modèles va influencer le nombre de threads

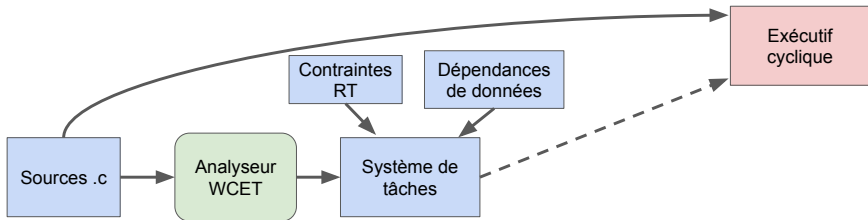
Implémentation

- Objectif : **Générer une application qui respecte les contraintes** présentes dans la spécification
- 2 grandes familles en temps-réel :
 - Solutions statiques : tout (ou presque) est déterminé **hors-ligne**
 - Produire un **ordonnancement statique** i.e. **un ordre d'exécution des tâches** (parfois aussi leurs dates de démarrage)
 - Soit sous la forme d'un **exécutif cyclique**, que l'on vérifie par la suite
 - Soit sous la forme d'une **table d'ordonnancement** qui contient la date de démarrage de chaque tâche, puis **générer le code correspondant** ou utiliser un **dispatcheur**
 - La vérification des propriétés temps-réel est **simple** : convient aux portions critiques
 - Solutions dynamiques : les décisions sont prises lors de l'exécution
 - Ecrire ou configurer un ordonnanceur qui prend les décisions
 - **Choisir une politique d'ordonnancement**
 - Produire les tâches
 - Implémentation plus simple, plus modulaire et plus robuste, mais moins de contrôle sur ce qu'il se passe : plus compliqué pour la certification

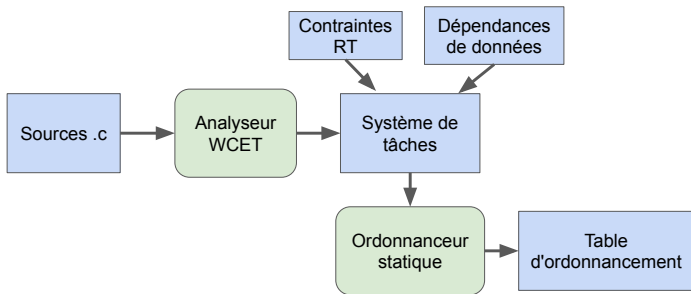
Solutions statiques



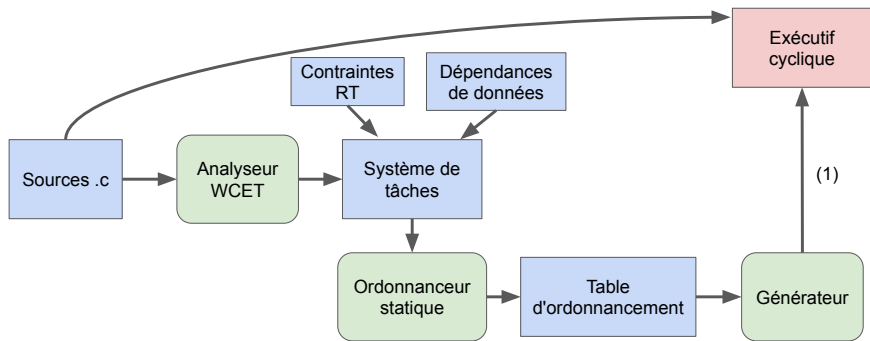
Solutions statiques



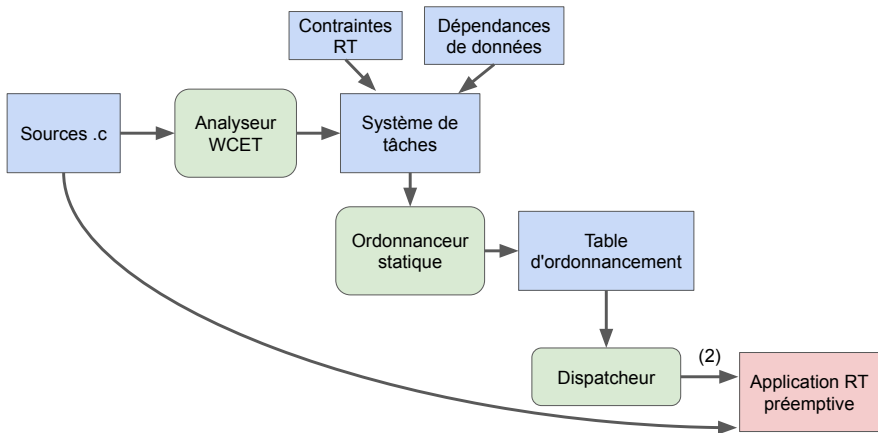
Solutions statiques



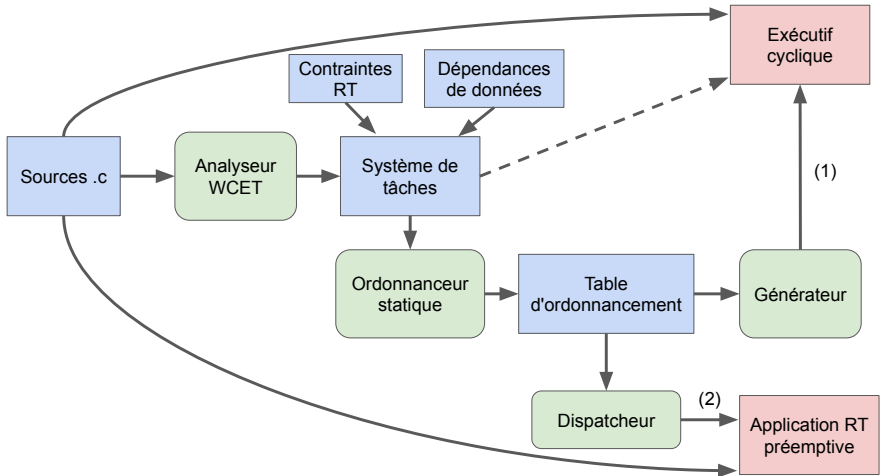
Solutions statiques



Solutions statiques



Solutions statiques



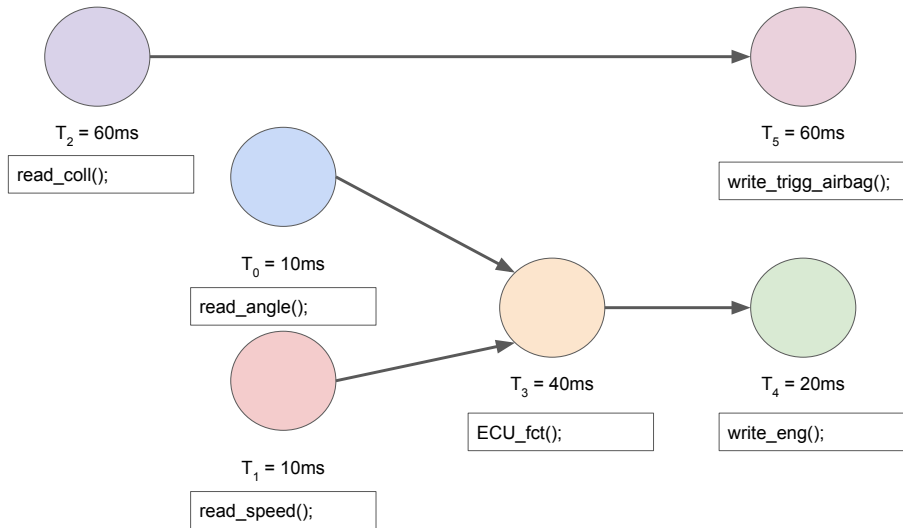
Solutions statiques

- Exécutif cyclique :
 - Une boucle infinie qui contient la séquence d'appels aux fonctions à répéter (cf. fonction de réaction)
 - Un seul thread

```
Loop:                                // Répétition infinie
    acquisitions_entrées();          // Echantillonnage capteurs
    calculs();
    écritures_sorties();             // Mise à jour actionneurs
```

- La boucle principale **fournit la période d'activation** au système entier
- On doit la **cadencer** sur la fréquence d'activation de la tâche **la plus rapide**, et ralentir les autres
 - En l'absence de synchronisations, la somme des WCETs des fonctions des tâches donne **le temps de réponse et la période** au pire cas

Solutions statiques



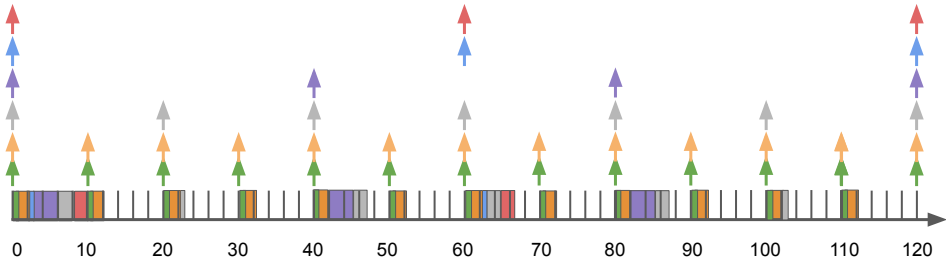
Solutions statiques

- On génère les appels dans un **ordre compatible avec les dépendances** du système, en commençant par les lectures des capteurs.
- e.g. $[\tau_0, \tau_1, \tau_2, \tau_3, \tau_4, \tau_5]$ ou $[\tau_2, \tau_1, \tau_0, \tau_3, \tau_5, \tau_4]$

```
Loop:                                // Répétition infinie
    read_angle();
    read_speed();
    if (i%6) read_coll();
    if (i%4) ECU_fct();
    if (i%2) write_eng();
    if (i%6) write_trigg_airbag();
    i++;
    wait_10ms();                     // Attente d'une interruption de
                                    // timer ou attente active
```

Solutions statiques

```
Loop:                                // Répétition infinie
  read_angle();
  read_speed();
  if (i%6) read_coll();
  if (i%4) ECU_fct();
  if (i%2) write_eng();
  if (i%6) write_trigg_airbag();
  i++;
  wait_10ms();                       // Attente d'une interruption de
                                     // timer ou attente active
```



Solutions statiques

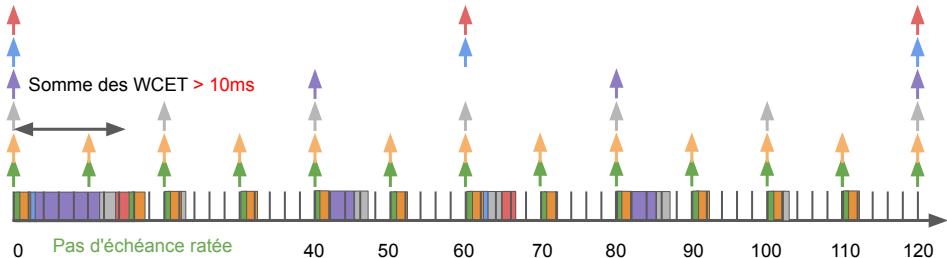
Pour les **exécutifs cycliques**

- **Critère suffisant :**
 - Somme des WCETs $<$ période de la boucle
 - Pas nécessaire
 - Le critère devient exact si toutes les tâches ont la même période
- **Critère exact :**
 - Construire l'ordonnancement pire-temps d'une **hyperpériode** et vérifier que chaque job **respecte son échéance**

Solutions statiques

Exécutifs cycliques : critère suffisant vs exact

```
Loop:                                // Répétition infinie
    read_angle();
    read_speed();
    if (i%6) read_coll();
    if (i%4) ECU_fct();
    if (i%2) write_eng();
    if (i%6) write_trigg_airbag();
    i++;
    wait_10ms();                     // Attente d'une interruption de
                                    // timer ou attente active
```

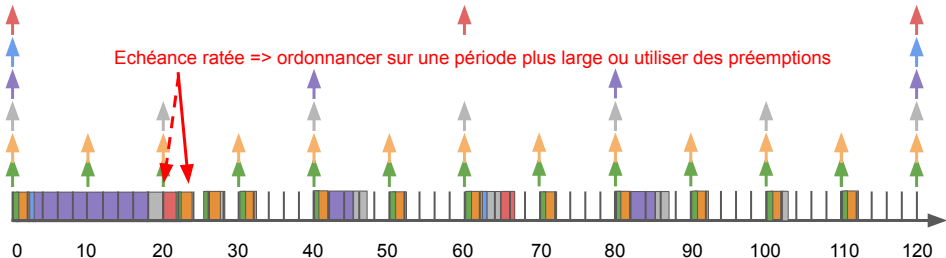


Solutions statiques

Exécutifs cycliques : problème des tâches lentes

```
Loop:                                // Répétition infinie
    read_angle();
    read_speed();
    if (i%6) read_coll();
    if (i%4) ECU_fct();
    if (i%2) write_eng();
    if (i%6) write_trigg_airbag();
    i++;
    wait_10ms();                     // Attente d'une interruption de
                                    // timer ou attente active
```

Echéance ratée => ordonnancer sur une période plus large ou utiliser des préemptions



Solutions statiques

Table d'ordonnancement

- Pour gérer les tâches longues **sans préemption**, on a besoin de **choisir les dates de démarrage** des différents jobs
- Le schéma de l'exécutif cyclique est **trop rigide**
- Pour cela on **construit** une **table d'ordonnancement** qui contient les dates de démarrage sur une **hyperpériode complète**
- On générera ensuite le code d'une hyperpériode qu'on exécutera **cycliquement**

Solutions statiques

Exécutifs cycliques : problème des tâches lentes

Echéance ratée => ordonnancer sur une période plus large ou utiliser des préemptions

Echéances respectées

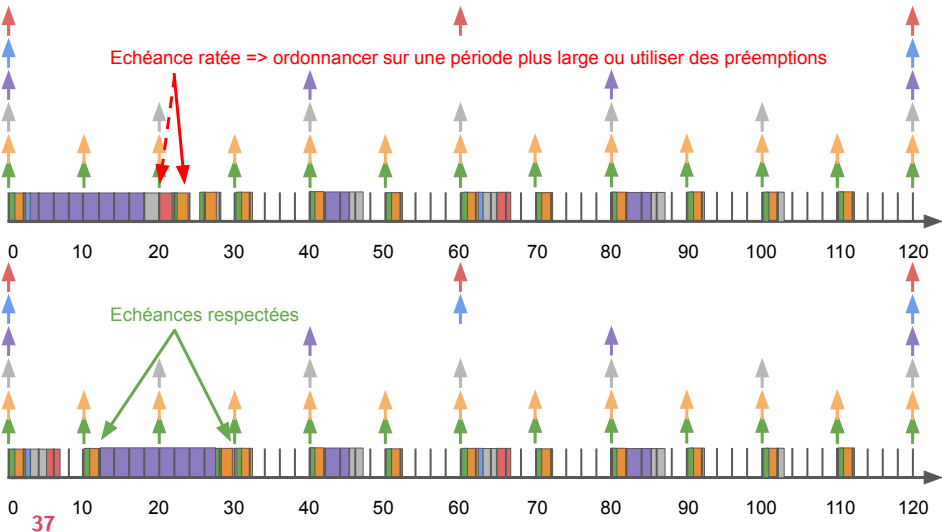


Table d'ordonnancement

- La construction de la table nécessite de choisir un **algorithme d'ordonnancement**
- Le problème est NP complet
 - Pour des petits jeux de tâches : à la main, **ILP** ou heuristiques
 - Pour des jeux plus gros : heuristiques

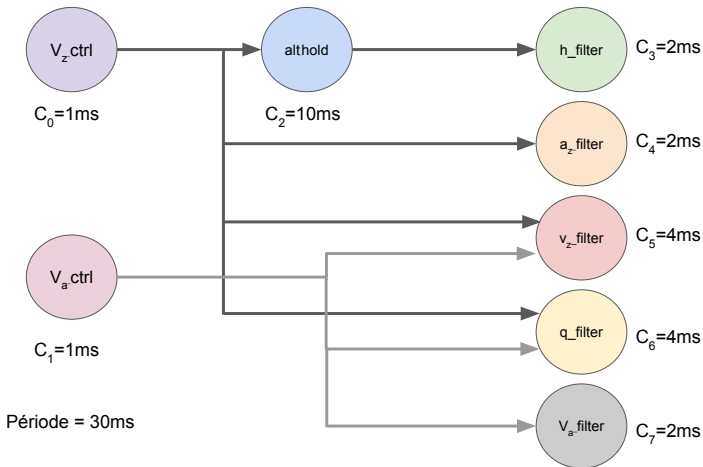
Solutions statiques

Table d'ordonnancement

- Quand il y a des communications explicites : **list scheduling** :
 - ① Construire une table vide de taille hyperpériode des tâches
 - ② Construire l'ensemble des jobs sur l'hyperpériode
 - ③ Ajouter à chaque job sa date d'activation
 - ④ Reporter le schéma communications sur les jobs
 - ⑤ Trier les jobs selon un ordre compatible avec les communications (créer une liste)
 - ⑥ Pour chaque job dans la liste, l'ordonnancer au plus tôt **après sa date d'activation**
 - ⑦ Si plusieurs choix sont possibles pour un job (e.g. multicoeur) : essayer toutes les possibilités, appliquer une fonction de coût, et choisir la solution la plus efficace. E.g. **minimisation du WCRT**
- Solution gloutonne
- Optimisations possibles :
 - Si le système est préemptif : si besoin découper un job en morceaux pour combler des trous
 - Si besoin, backtracker et modifier l'ordre de la liste

Solutions statiques

Exercice : construire la table d'ordonnancement de ce système de tâches **non-préemptives**, avec une technique de **list scheduling**



Solutions statiques

Etape 1 : Construire une table vide



Solutions statiques

Etape 5 : Trier les jobs



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_{z_filter}, v_{z_filter}, q_filter,$
 $v_{a_filter}]$

Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_{z_filter}, v_{z_filter}, q_filter,$
 $v_{a_filter}]$

Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_{z_filter}, v_{z_filter}, q_filter, v_{a_filter}]$

Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_z_filter, v_z_filter, q_filter, v_a_filter]$

Solutions statiques

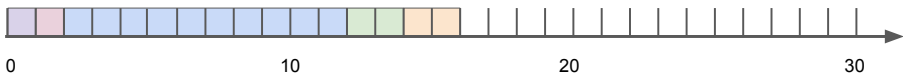
Etape 6 : Ordonnancer les jobs au plus tôt



[V_{z_ctrl} , V_{a_ctrl} , alt_hold , h_filter , a_{z_filter} , v_{z_filter} , q_filter , v_{a_filter}]

Solutions statiques

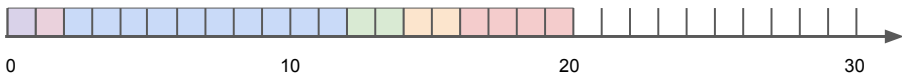
Etape 6 : Ordonnancer les jobs au plus tôt



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_{z_filter}, v_{z_filter}, q_filter, v_{a_filter}]$

Solutions statiques

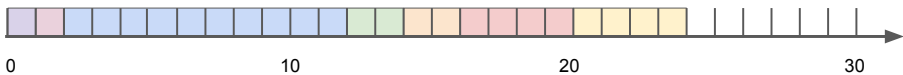
Etape 6 : Ordonnancer les jobs au plus tôt



[V_{z_ctrl} , V_{a_ctrl} , alt_hold, h_filter, a_z_filter, v_z_filter, q_filter, v_a_filter]

Solutions statiques

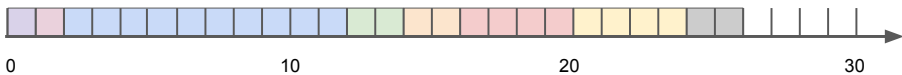
Etape 6 : Ordonnancer les jobs au plus tôt



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_{z_filter}, v_{z_filter}, q_filter, v_{a_filter}]$

Solutions statiques

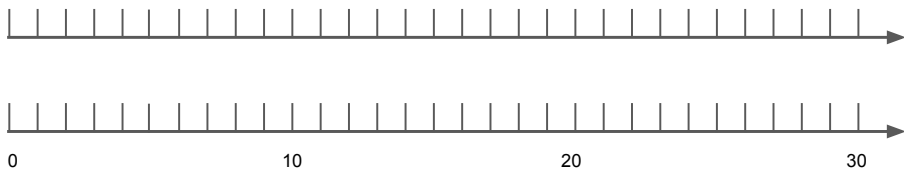
Etape 6 : Ordonnancer les jobs au plus tôt



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_{z_filter}, v_{z_filter}, q_filter, v_{a_filter}]$

Solutions statiques

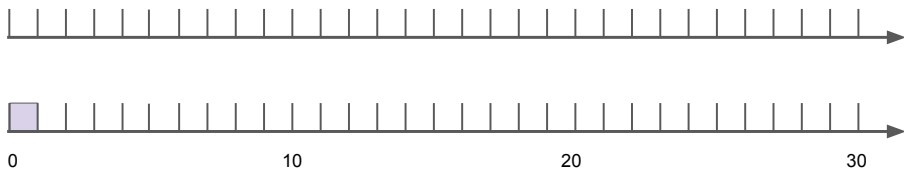
Même exercice mais avec un processeur dual-core



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_{z_filter}, v_{z_filter}, q_filter, v_{a_filter}]$

Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_{z_filter}, v_{z_filter}, q_filter, v_{a_filter}]$

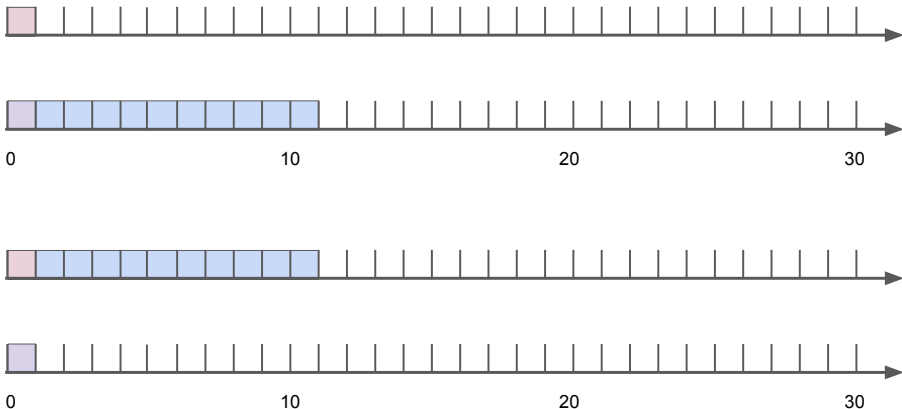
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt. En cas de multiples possibilités, on garde celle qui **minimise** le WCRT



Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt. En cas de multiples possibilités, on garde celle qui **minimise** le WCRT



Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt. En cas de multiples possibilités, on garde celle qui **minimise** le WCRT



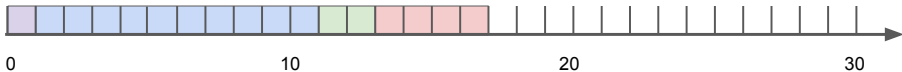
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt. En cas de multiples possibilités, on garde celle qui **minimise** le WCRT



Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt. En cas de multiples possibilités, on garde celle qui **minimise** le WCRT



Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt. En cas de multiples possibilités, on garde celle qui **minimise** le WCRT



Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt. En cas de multiples possibilités, on garde celle qui **minimise** le WCRT



Solutions statiques

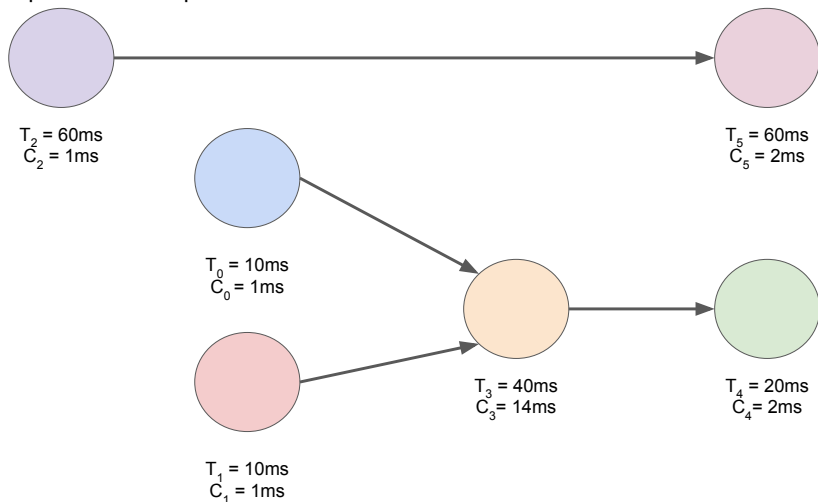
Etape 6 : Ordonnancer les jobs au plus tôt. En cas de multiples possibilités, on garde celle qui **minimise** le WCRT



$[V_{z_ctrl}, V_{a_ctrl}, alt_hold, h_filter, a_{z_filter}, v_{z_filter}, q_filter, v_{a_filter}]$

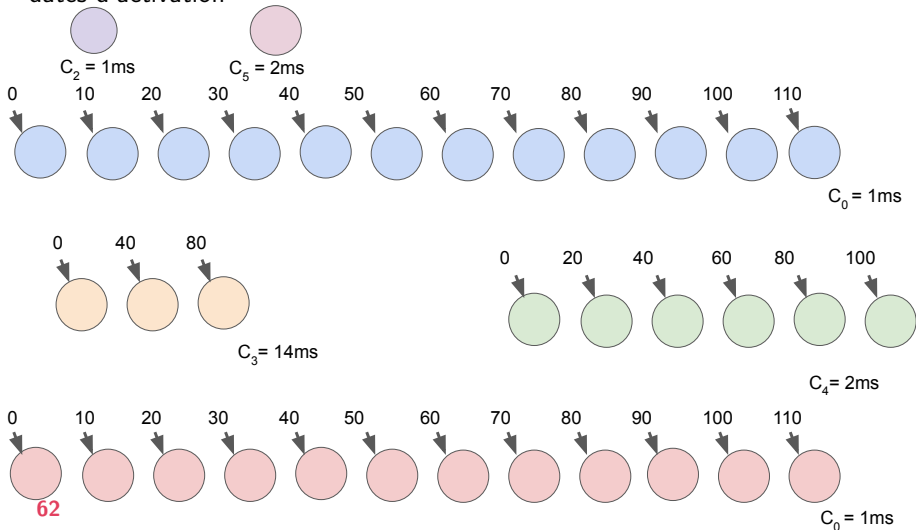
Solutions statiques

Exemple avec multipériode



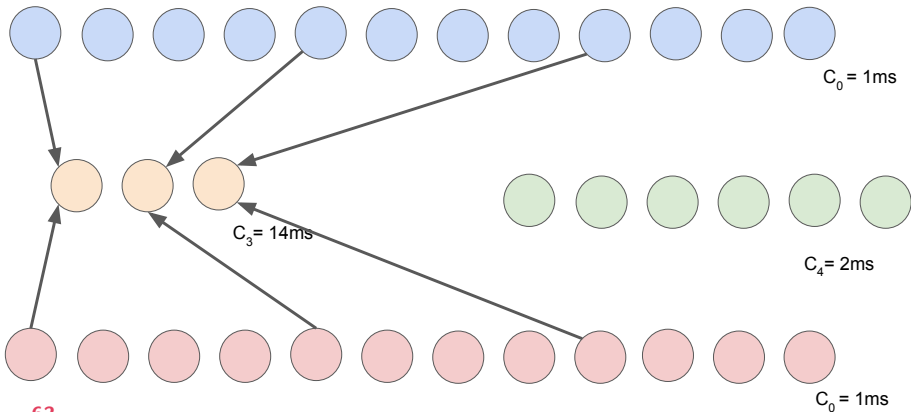
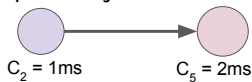
Solutions statiques

Etapes 2-3 : Construire l'ensemble des jobs sur l'hyperpériode + ajouter les dates d'activation



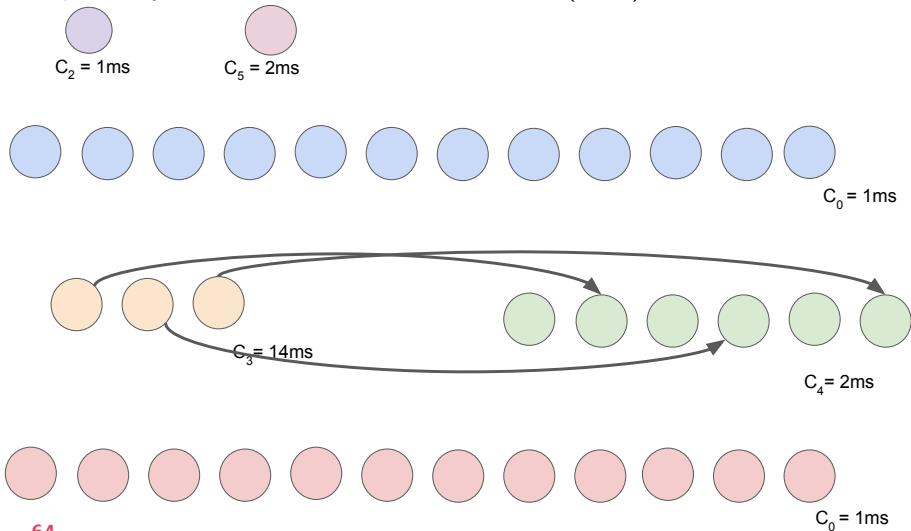
Solutions statiques

Etape 4 : Ajouter le schéma de communications



Solutions statiques

Etape 4 : Ajouter le schéma de communications (cont.)



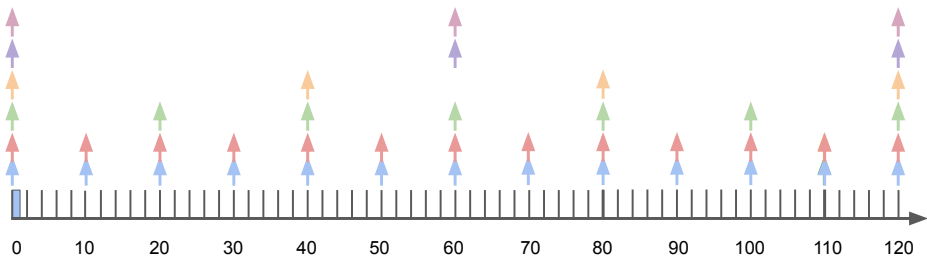
Solutions statiques

Etape 5 : Trier les jobs

$Jb_0, Jr_0, Jviolet, Jrose, Jv_0, Jb_1, Jr_1, Jj_0, Jb_2, Jr_2, Jb_3, Jr_3, Jb_4, Jr_4, \dots$

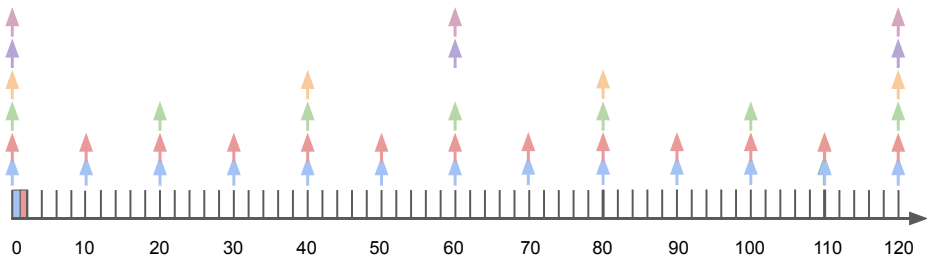
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



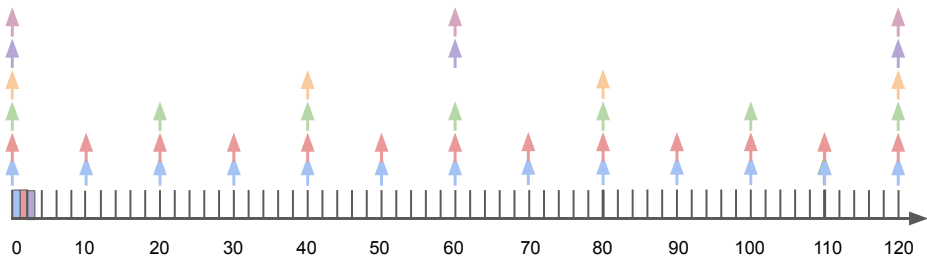
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



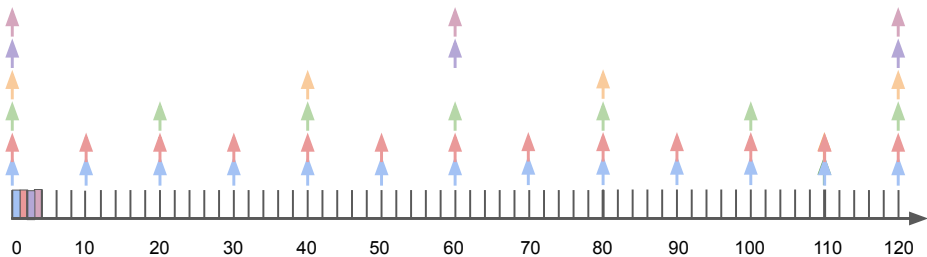
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



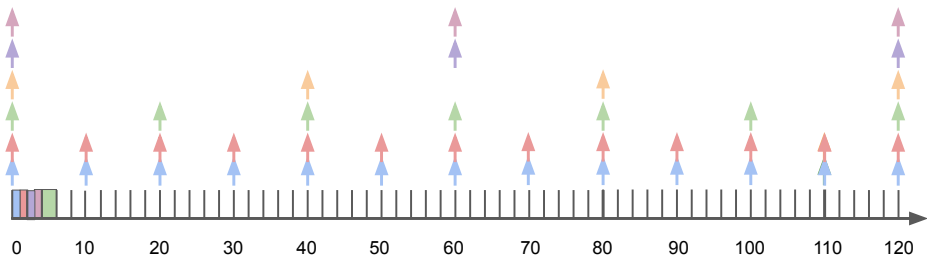
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



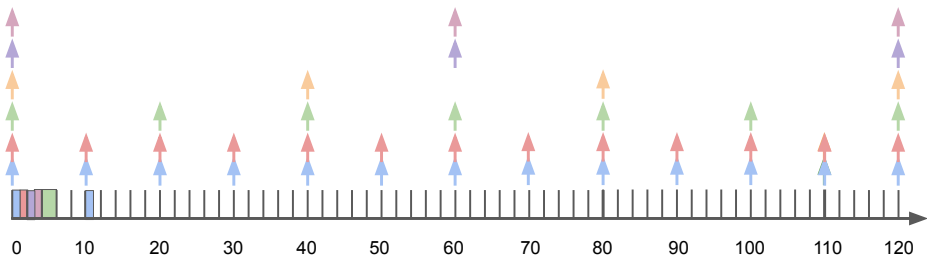
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



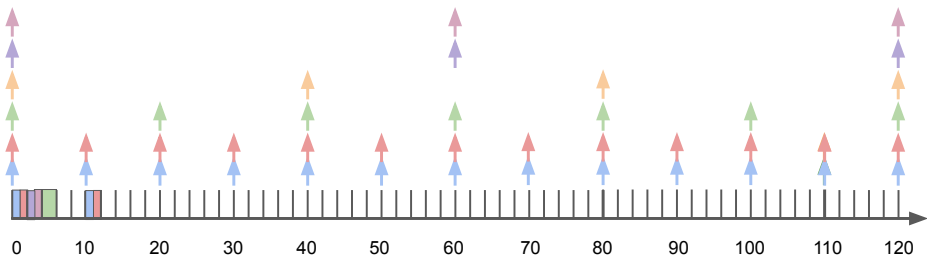
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



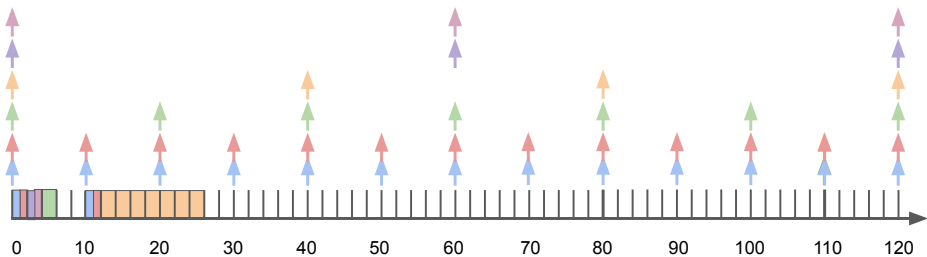
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



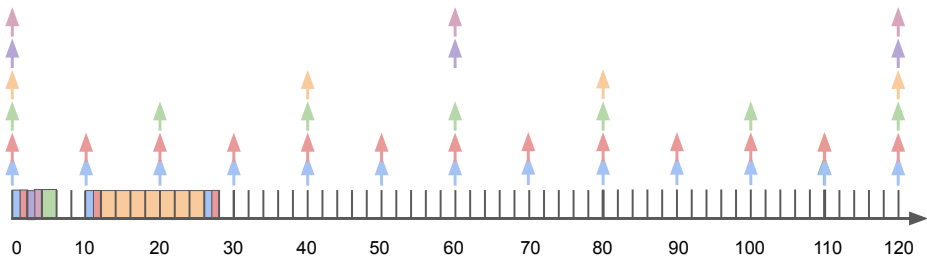
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



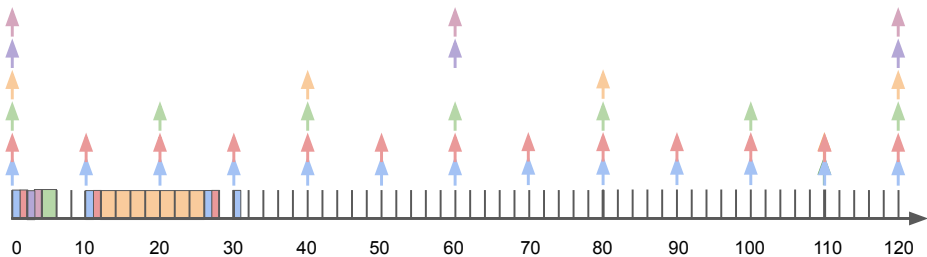
Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



Solutions statiques

Etape 6 : Ordonnancer les jobs au plus tôt



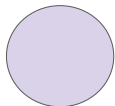
Solutions statiques

Table d'ordonnancement

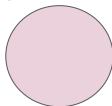
- Quand il n'y a pas de communication explicite : **simuler un ordonnanceur dynamique sur une hyperpériode**
 - e.g. RM, DM, EDF, LLF
 - Attention : plutôt efficaces pour les systèmes préemptifs
- Exemple **Rate monotonic** :
 - 1 Dessiner une table vide de taille hyperpériode des tâches
 - 2 Sélectionner la tâche de plus courte période
 - 3 Ordonnancer au plus tôt chaque job de cette tâche sur l'hyperpériode
 - 4 Si besoin, découper un job en morceaux pour le faire rentrer dans les espaces libres => préemptions
 - 5 Répéter 2 jusqu'à épuisement des tâches

Solutions statiques

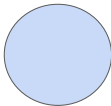
Exercice : construisez une table d'ordonnancement pour une période de ce système de tâches, **en utilisant l'algorithme Rate Monotonic préemptif**



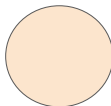
$$T_2 = 60\text{ms}$$
$$C_2 = 1\text{ms}$$



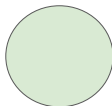
$$T_5 = 60\text{ms}$$
$$C_5 = 2\text{ms}$$



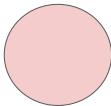
$$T_0 = 10\text{ms}$$
$$C_0 = 1\text{ms}$$



$$T_3 = 40\text{ms}$$
$$C_3 = 14\text{ms}$$



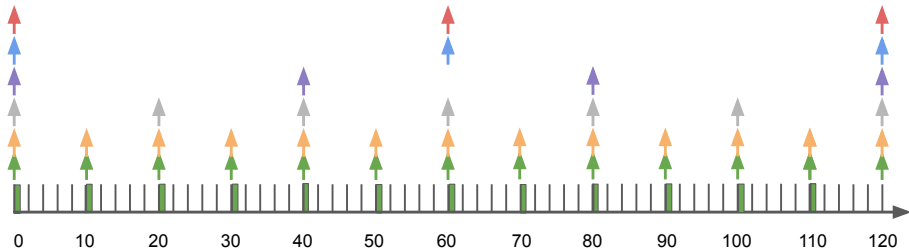
$$T_4 = 20\text{ms}$$
$$C_4 = 2\text{ms}$$



$$T_1 = 10\text{ms}$$
$$C_1 = 1\text{ms}$$

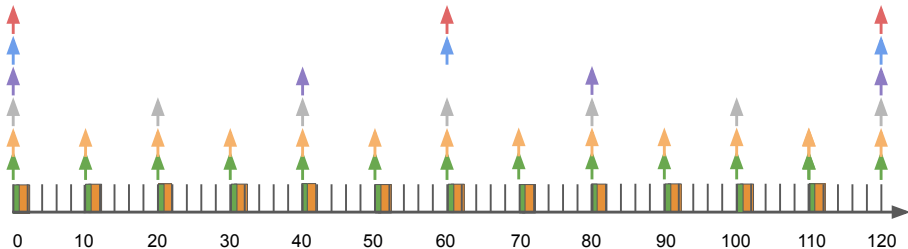
Solutions statiques

Exercice : construisez une table d'ordonnancement pour une période de ce système de tâches, **en utilisant l'algorithme Rate Monotonic préemptif**



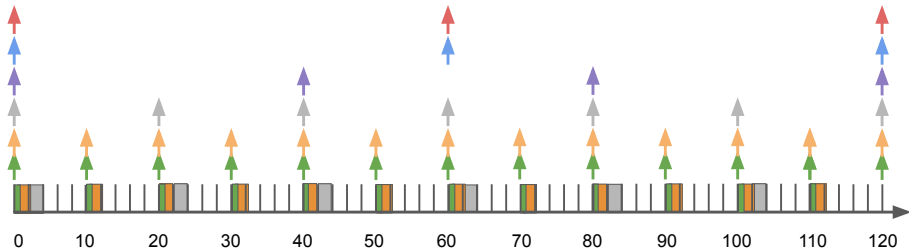
Solutions statiques

Exercice : construisez une table d'ordonnancement pour une période de ce système de tâches, **en utilisant l'algorithme Rate Monotonic préemptif**



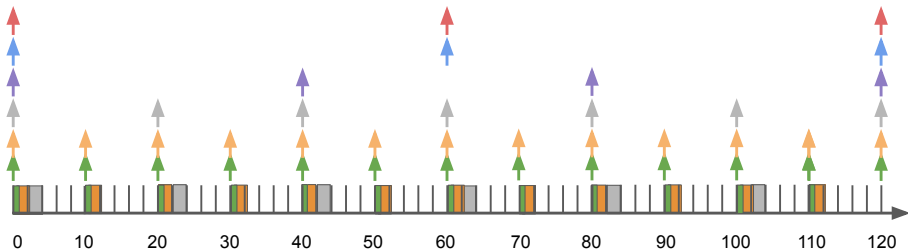
Solutions statiques

Exercice : construisez une table d'ordonnancement pour une période de ce système de tâches, **en utilisant l'algorithme Rate Monotonic préemptif**



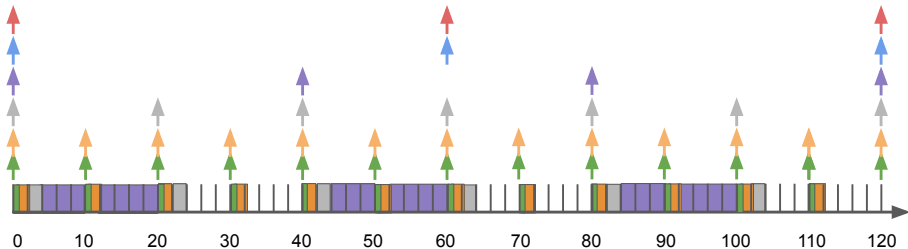
Solutions statiques

Exercice : construisez une table d'ordonnancement pour une période de ce système de tâches, **en utilisant l'algorithme Rate Monotonic préemptif**



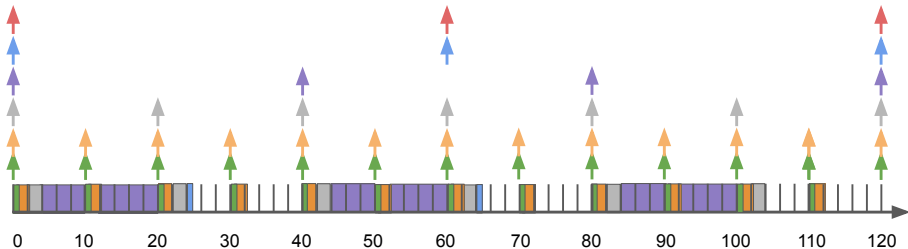
Solutions statiques

Exercice : construisez une table d'ordonnancement pour une période de ce système de tâches, **en utilisant l'algorithme Rate Monotonic préemptif**



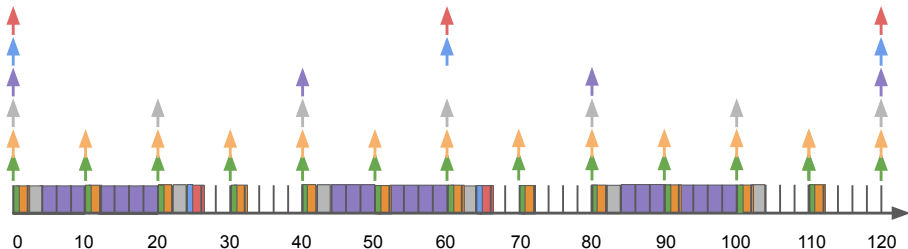
Solutions statiques

Exercice : construisez une table d'ordonnancement pour une période de ce système de tâches, **en utilisant l'algorithme Rate Monotonic préemptif**



Solutions statiques

Exercice : construisez une table d'ordonnancement pour une période de ce système de tâches, **en utilisant l'algorithme Rate Monotonic préemptif**



Solutions statiques

- Critère d'ordonnançabilité **suffisant** pour **RM/DM préemptif avec échéances implicites, sans section critique** :
 - $U = \sum_{i=1}^n C_i/T_i \leq n * (2^{1/n} - 1)$
 - $n \rightarrow +\infty : U \leq \ln(2) \simeq 0.69$
- Critère d'ordonnançabilité **exact** :
 - **Analyse WCRT** : pire-temps de réponse
 - Technique de calcul de **point fixe**
 - Idée : un job s'exécute dès qu'il est actif, mais peut être retardé par des jobs de priorité supérieures
 - Le WCRT est égal au budget d'exécution du job plus l'**interférence** des jobs de priorités supérieures

Solutions statiques

- Critère d'ordonnançabilité **suffisant** :
- $\tau_0 : T_0 = 10, C_0 = 3$
- $\tau_1 : T_1 = 5, C_1 = 1$
- $\tau_2 : T_2 = 5, C_2 = 2$
- $(\sum_{i=0}^2 C_i/T_i = 9/10 > 3 * (2^{1/3} - 1)(\simeq 0.77))$

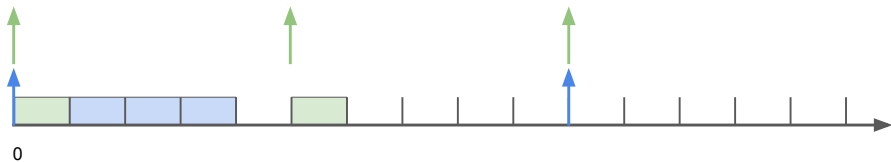
Solutions statiques

- Critère d'ordonnançabilité **exact** :
- $\tau_0 : T_0 = 10, C_0 = 3, \tau_1 : T_1 = 5, C_1 = 1, \tau_2 : T_2 = 5, C_2 = 2$
- τ_0 en isolation :



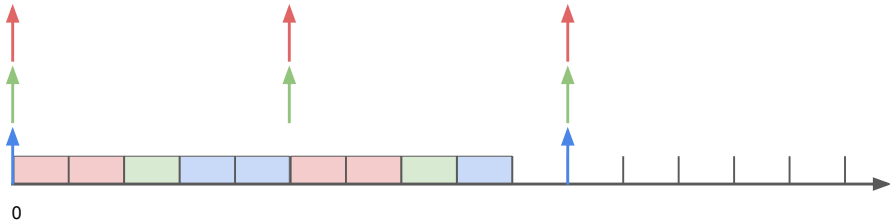
Solutions statiques

- Critère d'ordonnançabilité **exact** :
- $\tau_0 : T_0 = 10, C_0 = 3, \tau_1 : T_1 = 5, C_1 = 1, \tau_2 : T_2 = 5, C_2 = 2$
- τ_0 et τ_1 :



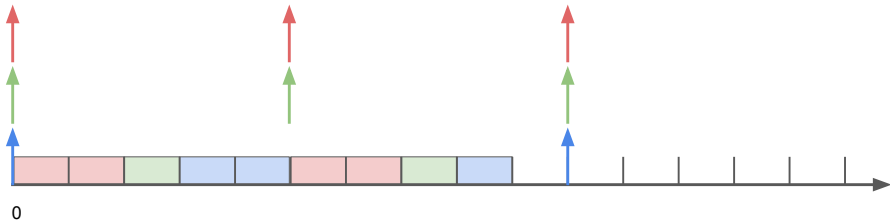
Solutions statiques

- Critère d'ordonnançabilité **exact** :
- $\tau_0 : T_0 = 10, C_0 = 3, \tau_1 : T_1 = 5, C_1 = 1, \tau_2 : T_2 = 5, C_2 = 2$
- τ_0, τ_1 et τ_2 :



Solutions statiques

- Critère d'ordonnançabilité **exact** :
- $\tau_0 : T_0 = 10, C_0 = 3, \tau_1 : T_1 = 5, C_1 = 1, \tau_2 : T_2 = 5, C_2 = 2$
- R_i/T_j donne le nombre de jobs de τ_j dans un intervalle de durée R_i



$$R_i = C_i + \sum_{j=i+1}^n \left\lceil \frac{R_i}{T_j} \right\rceil * C_j$$

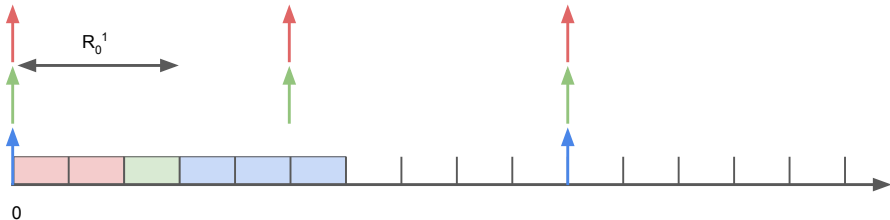
Solutions statiques

- Critère d'ordonnançabilité **exact** :
- $\tau_0 : T_0 = 10, C_0 = 3, \tau_1 : T_1 = 5, C_1 = 1, \tau_2 : T_2 = 5, C_2 = 2$
- Calcul de **point fixe** : $R_0^1 = C_0 = 3$



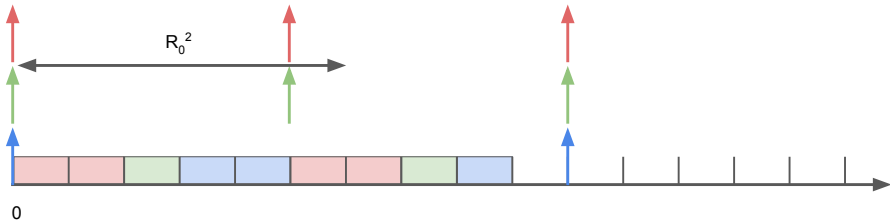
Solutions statiques

- Critère d'ordonnançabilité **exact** :
- $\tau_0 : T_0 = 10, C_0 = 3, \tau_1 : T_1 = 5, C_1 = 1, \tau_2 : T_2 = 5, C_2 = 2$
- Calcul de **point fixe** :
$$R_0^2 = C_0 + \lceil R_0^1 / T_1 \rceil * C_1 + \lceil R_0^1 / T_2 \rceil * C_2 = 3 + \lceil 3/5 \rceil * 1 + \lceil 3/5 \rceil * 2 = 6$$



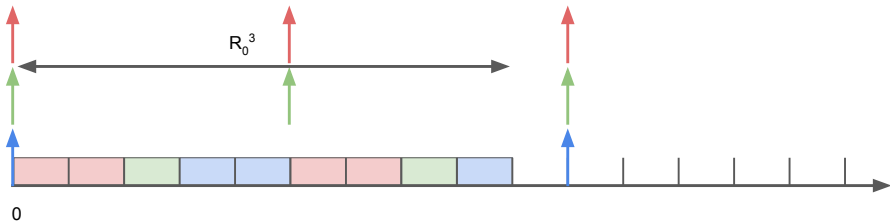
Solutions statiques

- Critère d'ordonnançabilité **exact** :
- $\tau_0 : T_0 = 10, C_0 = 3, \tau_1 : T_1 = 5, C_1 = 1, \tau_2 : T_2 = 5, C_2 = 2$
- Calcul de **point fixe** :
$$R_0^3 = C_0 + \lceil R_0^2/T_1 \rceil * C_1 + \lceil R_0^2/T_2 \rceil * C_2 = 3 + \lceil 6/5 \rceil * 1 + \lceil 6/5 \rceil * 2 = 9$$



Solutions statiques

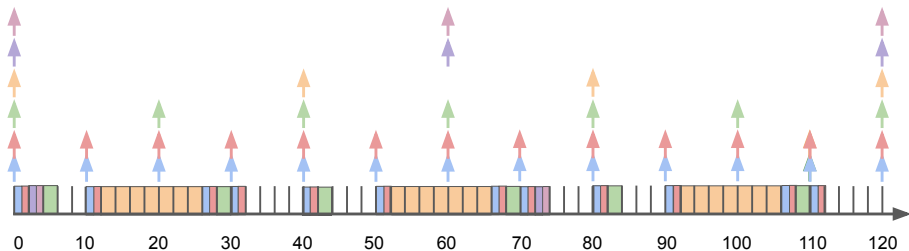
- Critère d'ordonnançabilité **exact** :
- $\tau_0 : T_0 = 10, C_0 = 3, \tau_1 : T_1 = 5, C_1 = 1, \tau_2 : T_2 = 5, C_2 = 2$
- Calcul de **point fixe** :
$$R_0^4 = C_0 + \lceil R_0^3 / T_1 \rceil * C_1 + \lceil R_0^3 / T_2 \rceil * C_2 = 3 + \lceil 9/5 \rceil * 1 + \lceil 9/5 \rceil * 2 = 9$$



Solutions statiques

- En l'absence de préemptions :
 - On génère un exécutif cyclique dont la période est l'hyperpériode des tâches
 - Les tâches de période plus **courtes** sont appelées plusieurs fois (plusieurs jobs) et ces appels sont **synchronisés** dans le temps
 - Si la tâche lit un capteur périodique, on s'assure qu'elle lit de nouvelles valeurs à chaque fois
 - Les mécanismes de synchronisation s'appuient sur des composants matériels
 - Echantillonnage d'un registre de performance (e.g. Time Stamp Counter register)
 - Echantillonnage d'un **timer**

Solutions statiques

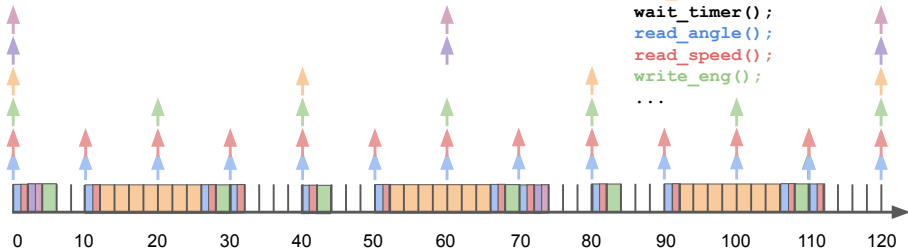


Solutions statiques

Option 1 :

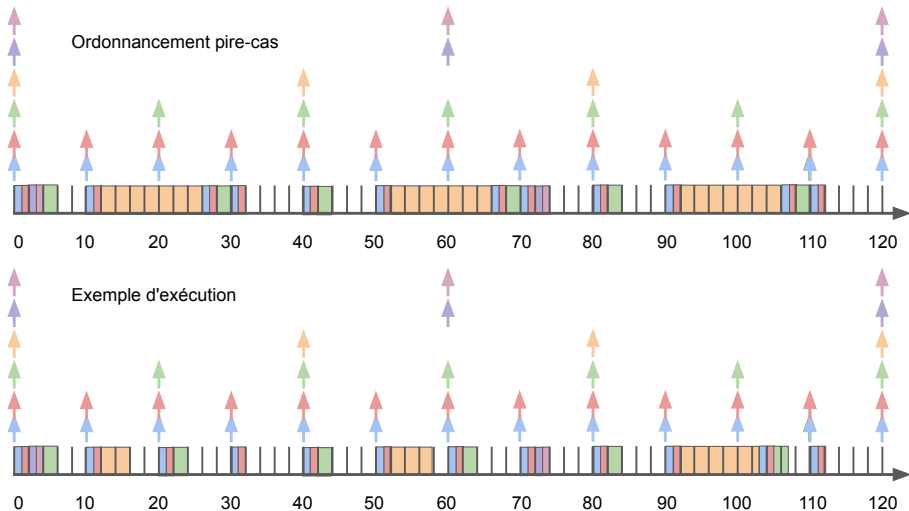
Synchroniser selon les **dates d'arrivée des données** (tâches harmoniques)

`wait_timer()` s'appuie sur un timer qui compte des périodes de **10ms**



```
loop:
    wait_timer();
    read_angle();
    read_speed();
    read_coll();
    write_trig_airbag();
    write_eng();
    wait_timer();
    read_angle();
    read_speed();
    ECU_fct();
    wait_timer();
    read_angle();
    read_speed();
    write_eng();
    ...
```

Solutions statiques

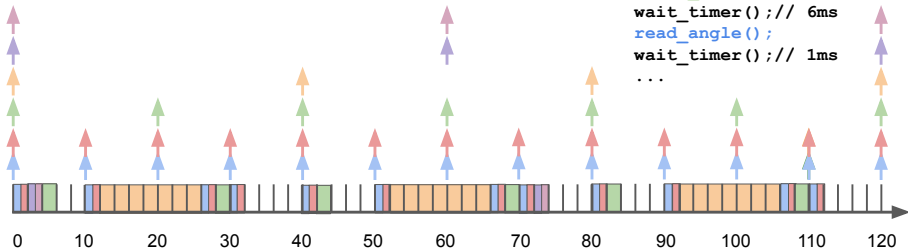


Solutions statiques

Option 2 :

Synchroniser selon les **dates dans la table**

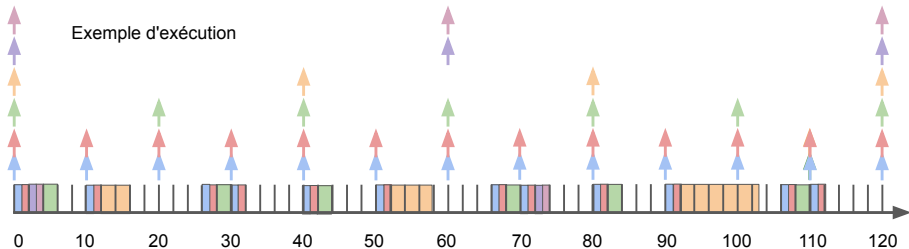
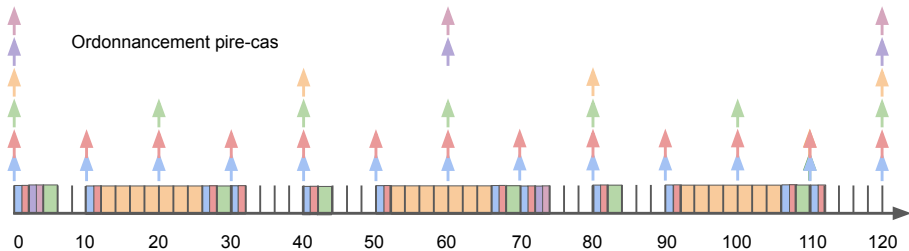
`wait_timer()` doit être **reconfigurée après chaque appel** pour compter des durées différentes



```
loop:
```

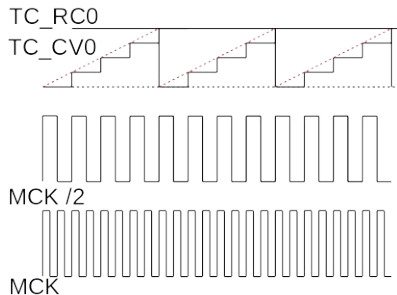
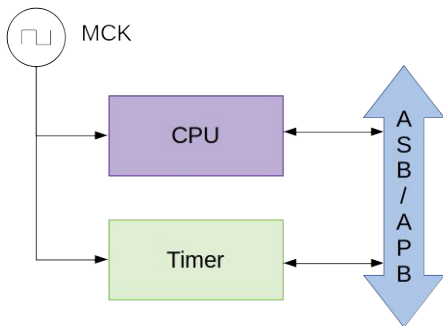
```
wait_timer();  
read_angle();  
wait_timer();// 1ms  
read_speed();  
wait_timer();// 1ms  
read_coll();  
wait_timer();// 1ms  
write_trig_airbag();  
wait_timer();// 1ms  
write_eng();  
wait_timer();// 6ms  
read_angle();  
wait_timer();// 1ms  
...
```

Solutions statiques



Solutions statiques

- Configuration du timer :
 - Compteur matériel (e.g. 16 bits sur le AT91SAM7S)
 - Incrémenté à une période multiple de l'horloge principale
 - Horloge cadencée à 48MHz sur le AT91
 - **Diviseurs** configurables



Solutions statiques

Configuration du timer :

- **Registre de commande CCR**
 - CLKDIS : **disable** bit
 - CLKEN : **enable** bit
 - SWTRG : **reset** bit
- **Plafond (16 bits) RC**
- **Compteur CV**
- **Registre de configuration CMR**
 - Diviseur de période DIV
 - Valeurs de diviseurs CLOCK1-5 (2, 8, 32, 128, 1024)
 - Remise à zéro de CV quand $CV = RC$ CPCTRG
- **Registre de status SR**
 - Bit à 1 quand CV atteint RC CPCS

Solutions statiques

Exemple : compter des secondes

① Trouver la valeur du diviseur DIV

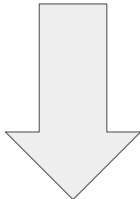
- $MCK = 48\text{MHz} \Rightarrow 48\,000\,000$ ticks par seconde
- RC mesure 16 bits $\Rightarrow$ compte jusqu'à **65 536**
- $48\,000\,000 / DIV \leq 65\,536 \Rightarrow DIV \geq 732,4 \Rightarrow \mathbf{DIV = CLOCK5 (1024)}$

② Fixer la valeur de RC (plafond)

- $48\,000\,000 / 1024 \leftrightarrow 1\text{s}$
- $\Rightarrow 1\text{s}$ équivaut à **CV = 46 875**
- On fixe RC à **46 875**

Solutions statiques

```
// Initialisation du timer
TC0_CCR = TC_CLKDIS; /* Désarmer le timer */
TC0_CMR = TC_CLOCK5  /* Diviseur 1024 */
          | TC_CPCTRG; /* RaZ quand CV atteint RC */
TC0_RC = 46875;       /* 1s */
TC0_CCR = TC_CLKEN;   /* Armer le timer */
TC0_CCR = TC_SWTRG;   /* Réinitialiser le timer */
```



```
// wait_timer()
while(!(TC0_SR & TC0_CPCS)); // Attente du bit CPCS dans le SR
```

Solutions statiques

Le timer doit compter des périodes de **10ms**

Exercice : **trouver la configuration des registres du timer**

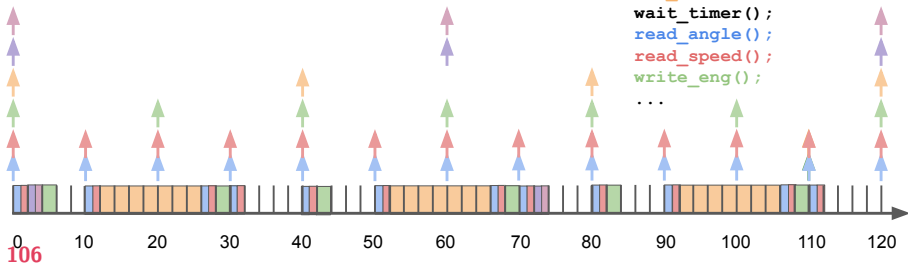
Option 1 :

Synchroniser selon les **dates d'arrivée des données** (tâches harmoniques)

`wait_timer()` s'appuie sur un timer qui compte des périodes de **10ms**

loop:

```
wait_timer();  
read_angle();  
read_speed();  
read_coll();  
write_trig_airbag();  
write_eng();  
wait_timer();  
read_angle();  
read_speed();  
ECU_fct();  
wait_timer();  
read_angle();  
read_speed();  
write_eng();  
...
```



Solutions statiques

```
// Initialisation du timer
TC0_CCR = TC_CLKDIS; /* Désarmer le timer */
TC0_CMR = TC_CLOCK2 /* Diviseur 8 */
          | TC_CPCTRG; /* RaZ quand CV atteint RC */
TC0_RC = 60000; /* 10ms */
TC0_CCR = TC_CLKEN|TC_SWTRG; /* Armer et réinitialiser le timer */
```

```
// Loop
while(1){
    read_angle();
    read_speed();
    read_coll();
    write_trig_airbag();
    write_eng();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 10ms */
    read_angle();
    read_speed();
    ECU_fct();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 20ms */
    ...
    while(!(TC0_SR & TC_CPCS)); /* Après la date 110ms */
    read_angle();
    read_speed();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 120ms */
}
```

SYNCHRONISATION SUR LES DATES D'ARRIVÉE DES DONNÉES

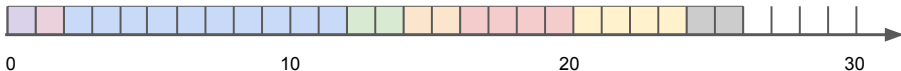
Solutions statiques

```
// Initialisation du timer
TC0_CCR = TC_CLKDIS; /* Désarmer le timer */
TC0_CMR = TC_CLOCK3 /* Diviseur 32 */
          | TC_CPCTR; /* RaZ quand CV atteint RC */
TC0_RC = 1500; /* 1ms */
TC0_CCR = TC_CLKEN|TC_SWTRG; /* Armer et réinitialiser le timer */

// Loop
while(1){
    read_angle();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 1ms */
    read_speed();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 2ms */
    read_coll();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 3ms */
    write_trig_airbag();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 4ms */
    TC0_RC = 9000; /* Attente 6ms */
    write_eng();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 10ms */
    TC0_RC = 1500; /* Attente 1ms */
    read_angle();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 11ms */
    read_speed();
    while(!(TC0_SR & TC_CPCS)); /* Après la date 12ms */
    TC0_RC = 21000; /* Attente 14ms */
    ECU_fct();
    ...
    while(!(TC0_SR & TC_CPCS)); /* Après la date 120ms */
}
```

Solutions statiques

Exercice : écrire le code correspondant à cet ordonnancement, en assurant que chaque tâche démarre à la date prévue dans la table



`[Vz_ctrl, Va_ctrl, alt_hold, h_filter, az_filter, vz_filter, q_filter, va_filter]`

Solutions statiques

Exercice : écrire le code correspondant à ce système

- $\tau_0 : T_0 = 10ms, C_0 = 2ms$
- $\tau_1 : T_1 = 50ms, C_1 = 15ms$
- $\tau_2 : T_2 = 100ms, C_2 = 16ms$

Solutions statiques

Exercice : écrire le code correspondant à ce système

- $\tau_0 : T_0 = 20ms, C_0 = 5ms$
- $\tau_1 : T_1 = 30ms, C_1 = 5ms$
- $\tau_2 : T_2 = 60ms, C_2 = 10ms$

Solutions statiques

- Utilisation d'un dispatcher
 - La **table d'ordonnement** est transformée en struct C
 - Des **interruptions** appellent le dispatcher
 - Le dispatcher lit la table et **appelle les fonctions** des tâches
 - Le dispatcher maintient une représentation interne de la **date courante**
 - Le WCET de la routine d'interruption et du dispatcher doivent être inclus dans le WCET des tâches

Solutions statiques

- En cas de **préemptions**, il faut sauvegarder/restaurer le **contexte** de la tâche préemptée
 - Le **coût** de la sauvegarde/restauration doit être pris en compte dans le WCET
 - Sections critiques => attention aux **inversions de priorités**

- Time Triggered Architecture (TTA)
 - Les **timers** sont les seules sources d'interruption
 - Les préemptions se produisent suite aux **interruptions** des timers
 - TTA est un sous ensemble des systèmes "**event-triggered**", plus simple à gérer car les interruptions sont **périodiques**

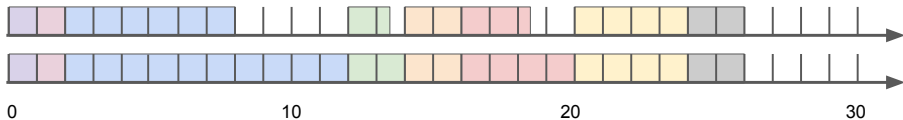
Solutions statiques

```
typedef struct ORDER {
    unsigned int date;           // Comptée en nombre d'interruptions
    unsigned char preemption;    // 0-> appel, 1-> préemption, 2-> retour de préemption
    void (* address) ();        // Fonction à appeler
} order_t;

typedef struct SCHEDULE {
    int index;                  // Indice courant dans la table
    int size_index;             // Nombre d'entrées dans la table
    int size_duration;          // Nombre d'interruptions avant la prochaine hyperpériode
    order_t *table;             // Table d'ordo
} schedule_t;
```

Exemple: timer produisant des interruptions toutes les ms (et pas de préemption)

```
schedule_t schedule;
order_t st[8] = {0, 0, f}, {1, 0, g}, {2, 0, h}, {12, 0, i}, {14, 0, j}, {16, 0, k}, {20, 0, l}, {24, 0, m};
schedule.index = 0; // Démarrer au début de la table
schedule.size_index = 8;
schedule.size_duration = 30;
schedule.table = &st[0];
```



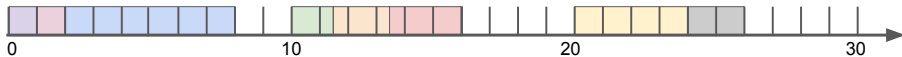
Solutions statiques

```
typedef struct ORDER {
    unsigned int date;          // Comptée en nombre d'interruptions
    unsigned char preemption;   // 0-> appel, 1-> préemption, 2-> retour de préemption
    void (* address) ();       // Fonction à appeler
} order_t;

typedef struct SCHEDULE {
    int index;                  // Indice courant dans la table
    int size_index;             // Nombre d'entrées dans la table
    int size_duration;         // Nombre d'interruptions avant la prochaine hyperpériode
    order_t *table;            // Table d'ordo
} schedule_t;
```

Exemple: timer produisant des interruptions toutes les 10ms (et pas de préemption).

```
schedule_t schedule;
order_t st[3] = {0, 0, fgh}, {1, 0, ijk}, {2, 0, lm};
schedule.index = 0; // Démarrer au début de la table
schedule.size_index = 3;
schedule.size_duration = 3;
schedule.table = &st[0];
```



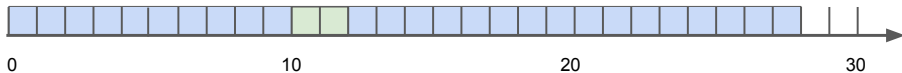
Solutions statiques

```
typedef struct ORDER {
    unsigned int date;           // Comptée en nombre d'interruptions
    unsigned char preemption;    // 0-> appel, 1-> préemption, 2-> retour de préemption
    void (* address) ();        // Fonction à appeler
} order_t;

typedef struct SCHEDULE {
    int index;                   // Indice courant dans la table
    int size_index;              // Nombre d'entrées dans la table
    int size_duration;           // Nombre d'interruptions avant la prochaine hyperpériode
    order_t *table;              // Table d'ordo
} schedule_t;
```

Exemple: timer produisant des interruptions toutes les 2ms (et une préemption).

```
schedule_t schedule;
order_t st[3] = {0, 0, h}, {5, 1, i}, {6, 2, h};
schedule.index = 0; // Démarrer au début de la table
schedule.size_index = 3;
schedule.size_duration = 15;
schedule.table = &st[0];
```



Solutions statiques

- Gestion des interruptions par timer sur ARM
- **Table d'exceptions ARM** : adresse 0x0
- Les interruptions redirigent vers l'**AIC** (MUX d'IRQ)

Solutions statiques

```
int main(){
    // Initialiser le timer

    TC0_CCR = TC_CLKDIS;
    TC0_CMR = TC_CLOCK3 | TC_CPCTRG; /* Diviseur et RaZ quand CV = RC */
    TC0_RC = 3000; /* 2ms */
    TC0_IER = TC_CPCS; /* Générer une interruption quand CV = RC */

    // Configurer l'AIC

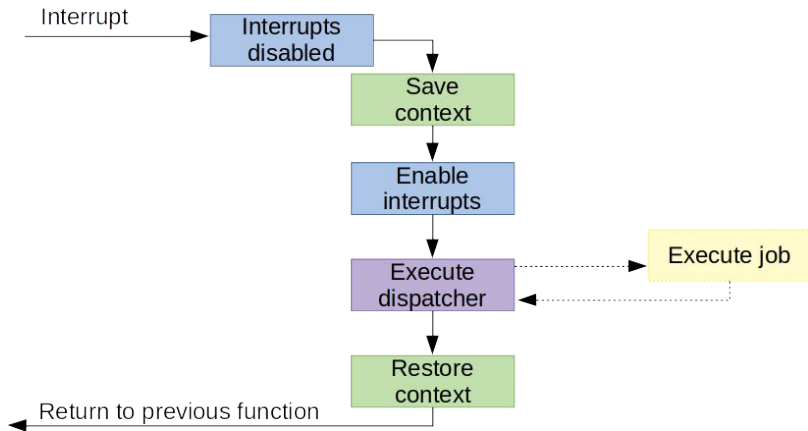
    AIC_SVR[ID_TC0] = (uint32_t)handle_tc0; /* Configurer la routine d'interruption */
    AIC_SMR[ID_TC0] = 0; /* Priorité de l'interruption */
    AIC_ICCR = 1 << ID_TC_0; /* Nettoyer les interruptions potentielles */
    AIC_IECR = 1 << ID_TC_0; /* Activer les interruptions dans l'AIC */
    TC0_CCR = TC_SWTRG | TC_CLKEN; /* Activation TC0 et RaZ TC0 */

    schedule_t schedule;
    order_t st[3] = {0, 0, h}, {5, 1, i}, {6, 1, h};
    schedule.index = 0; // Démarrer au début de la table
    schedule.size_index = 3;
    schedule.size_duration = 15;
    schedule.table = &st[0];

    while(1);
    return 0;
}
```

Solutions statiques

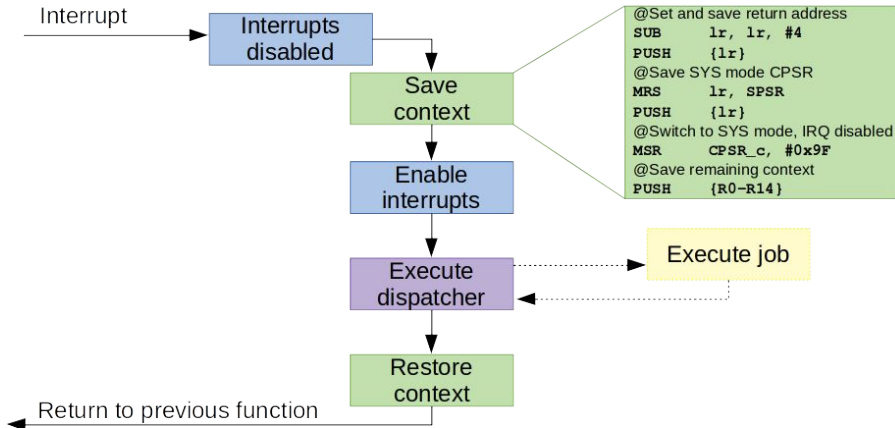
Routine d'interruption :



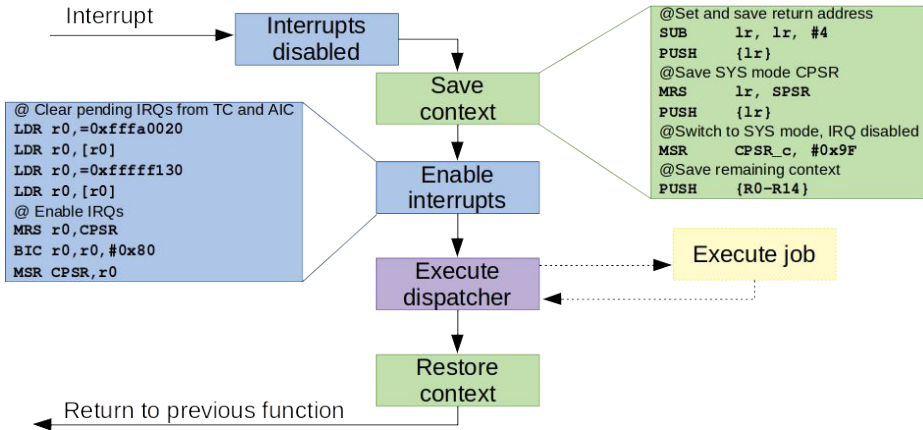
Solutions statiques

- Au moment de l'interruption => **bascule en mode IRQ**
- Les interruptions sont **désactivées**
- Registres spéciaux : SP_IRQ, LR_IRQ et SPSR_IRQ
 - Le PC du mode SYS est dans LR_IRQ
 - Le CPSR du mode SYS est dans SPSR_IRQ
- En cas de préemptions "en nid" => On sauve **l'adresse de retour** sur la pile de l'IRQ
 - `stack_irq <- LR_IRQ -4`
- On sauve le **SPSR** sur la pile de l'IRQ
- On bascule en mode **SYS**
- On sauve le contexte (**r0-r14**) sur la pile de l'IRQ
- On rétablit les interruptions et on **branche vers le dispatcher**
- **Au retour du branchement** : on restore le contexte
- On repart de l'interruption
 - On bascule en mode **IRQ**
 - On restore le SPSR depuis la pile IRQ
 - On restore LR_IRQ depuis la pile IRQ
 - `PC <- LR` et on copie le SPSR (IRQ) dans le CPSR (SYS)

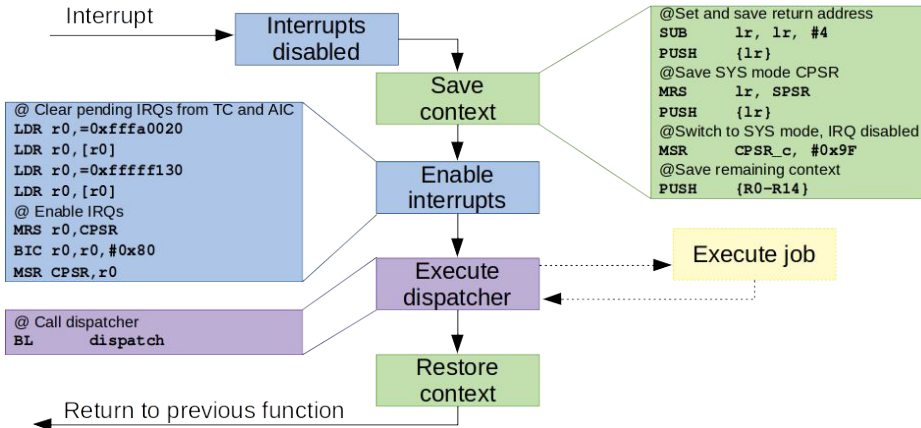
Solutions statiques



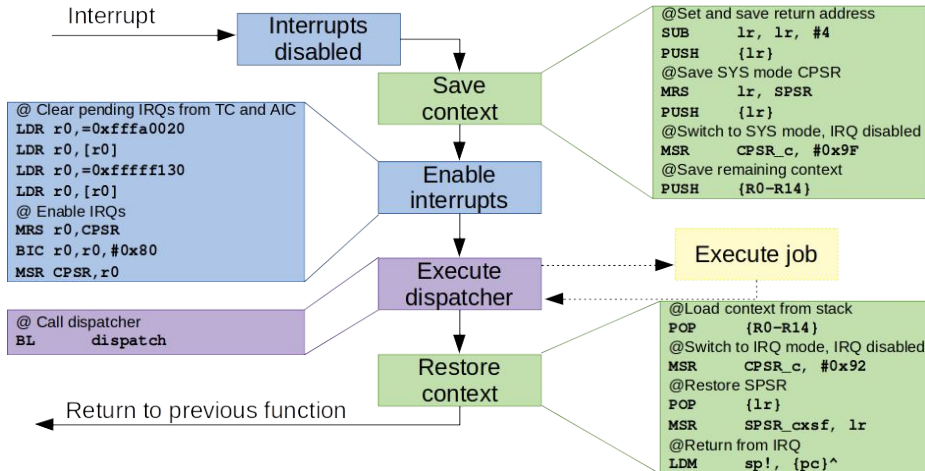
Solutions statiques



Solutions statiques



Solutions statiques

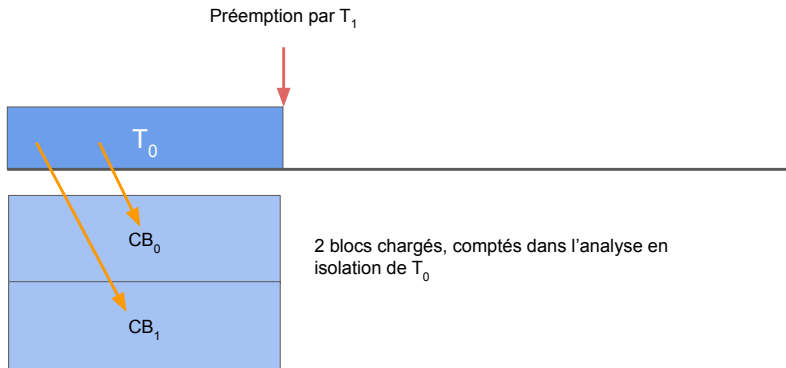


Solutions statiques

- Dispatcher
 - ① Gérer la représentation interne du temps (incrément et/ou RaZ)
 - ② Lire la table et si besoin appeler la fonction correspondante (la date correspond et l'entrée de la table est un appel de fonction)
 - ③ Gérer l'indice (incrément et/ou RaZ et/ou pas de changement)

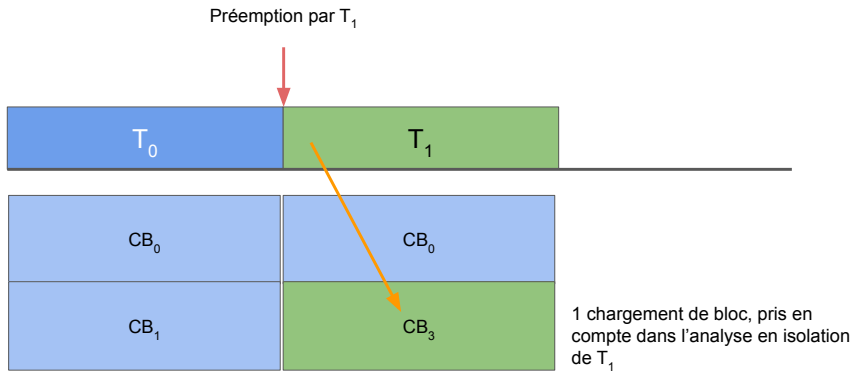
Problèmes ouverts

Préemptions et CRPD



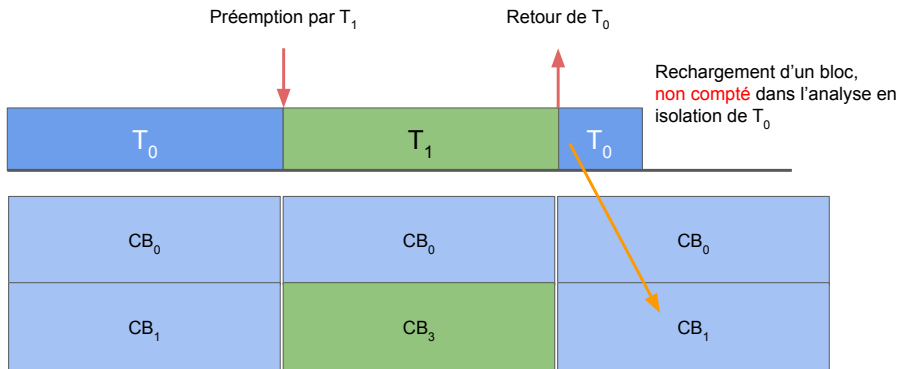
Problèmes ouverts

Préemptions et CRPD



Problèmes ouverts

Préemptions et CRPD



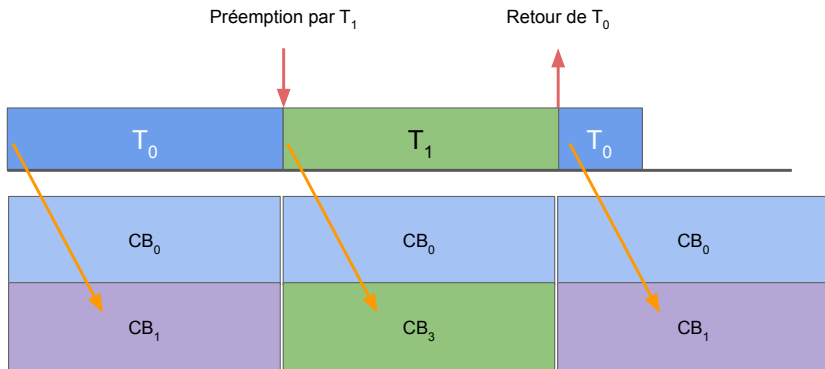
Problèmes ouverts

Préemptions et CRPD

- Solution par interprétation abstraite :
- Calcul des blocs **dont la tâche préemptée a besoin après leur chargement** : UCB (Useful Cache Blocks)
 - Blocks MAY à un point donné, et MUST lors d'un autre accès subséquent
- Calcul des blocs **potentiellement chargés par la tâche préemptrice** : ECB (Evicting Cache Blocks)
 - Blocks MAY n'importe où dans la tâche
- Croisement des blocs UCB et ECB, suivant la politique du cache

Problèmes ouverts

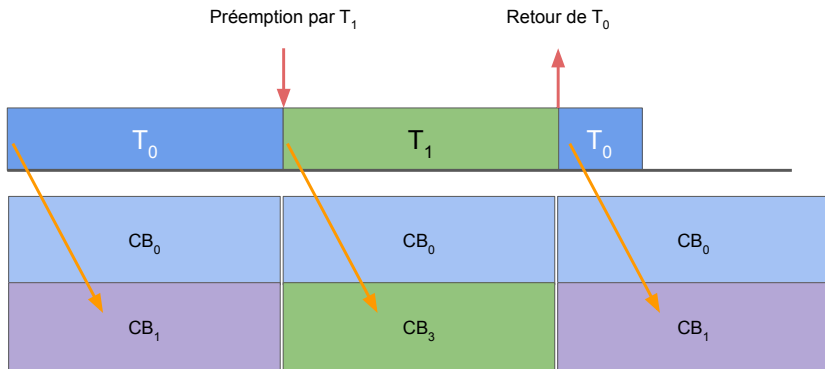
Préemptions et CRPD



CB_1 est un UCB à partir de son chargement

Problèmes ouverts

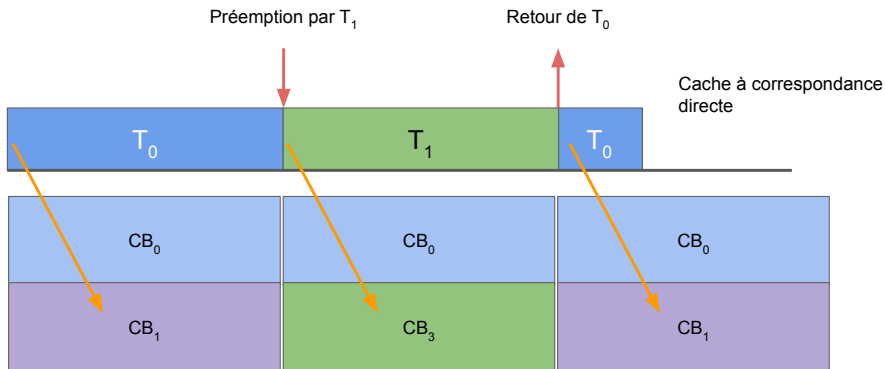
Préemptions et CRPD



CB_3 est un ECB

Problèmes ouverts

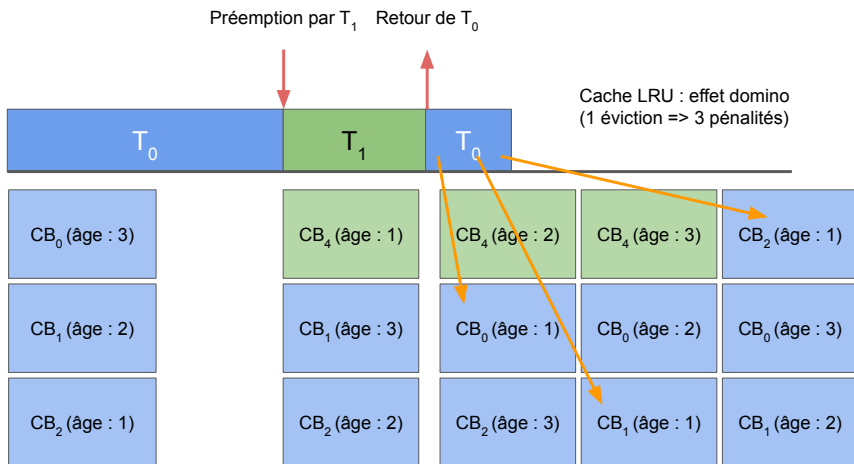
Préemptions et CRPD



CB_1 et CB_3 sont sur le même ensemble dans le cache => une pénalité doit être ajoutée

Problèmes ouverts

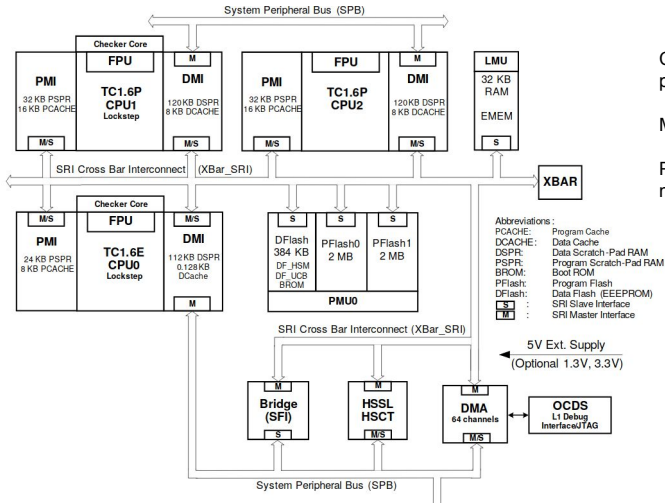
Préemptions et CRPD



Ensemble n

Problèmes ouverts

Interférence multi-coeurs



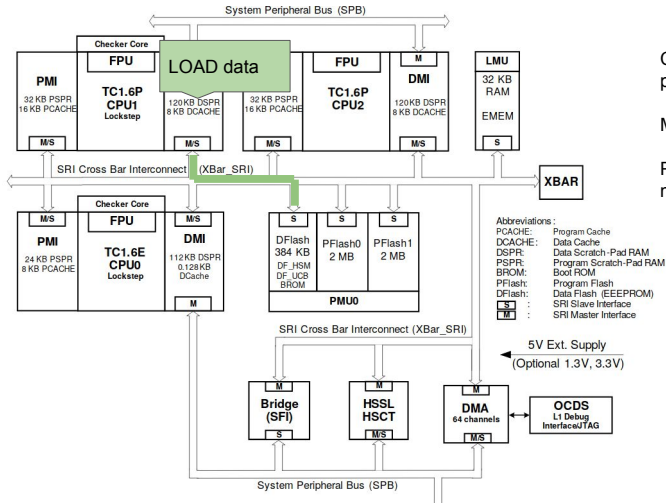
Coeurs indépendants du point de vue logique

MAIS :

Partage de composants matériels

Problèmes ouverts

Interférence multi-cœurs



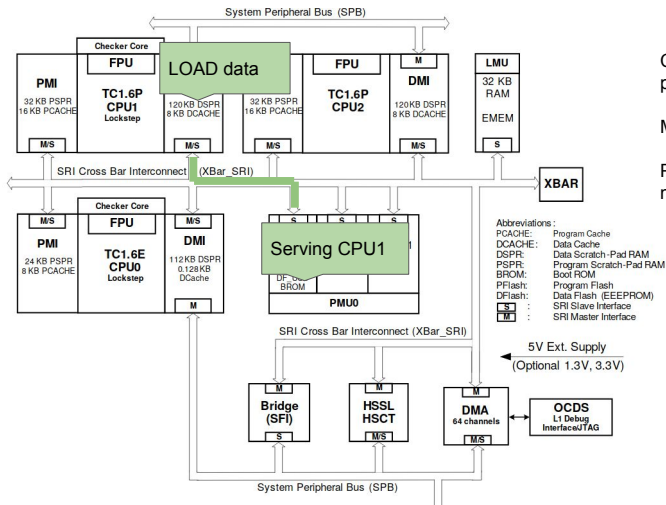
Cœurs indépendants du point de vue logique

MAIS :

Partage de composants matériels

Problèmes ouverts

Interférence multi-coeurs



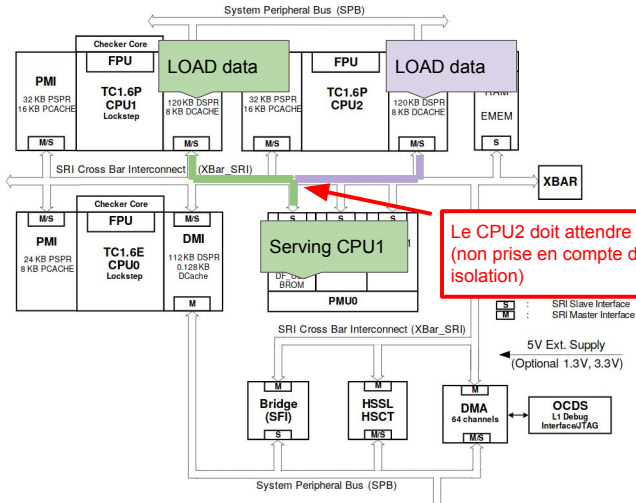
Coeurs indépendants du point de vue logique

MAIS :

Partage de composants matériels

Problèmes ouverts

Interférence multi-coeurs



Coeurs indépendants du point de vue logique

MAIS :

Partage de composants matériels

Le CPU2 doit attendre => interférence
(non prise en compte dans l'analyse WCET en isolation)

Problèmes ouverts

Interférence multi-coeurs

- Solutions analytiques : modéliser tous les accès possibles de toutes les tâches du système $\Rightarrow$ combinatoire
- Solutions compositionnelles : amalgamer les accès à la granularité des tâches et appliquer des pénalités quand 2 tâches sont ordonnancées en parallèle
- Solution compilation : modèle 3 phases

Problèmes ouverts

Interférence multi-coeurs : modèle 3 phases

- Les tâches sont créées en 3 phases : Acquisition, Exécution, Restitution
- Les accès mémoire sont groupés dans les phases A (copies distant -> local) et R (local -> distant)
- On crée un ordonnancement sans interférence

